

青 岛 科 技 大 学

移动软件开发课程论文

题 目 智慧商城

计算 213 刘 洋 2108010316

信息学院 学院 计算机科学与技术 专业

2024 年 7 月 7 日

智慧商城

摘要

智慧商城小程序是一款基于微信小程序框架开发的电商平台，旨在为用户提供便捷、高效的购物体验。项目的开发背景源于电子商务的快速发展和智能手机的普及，用户对移动购物的需求日益增加。然而，现有的电商平台在用户体验、个性化推荐、虚拟试穿等方面存在不足。基于此，本项目结合了 AR 技术、AI 算法和高并发处理等前沿技术，旨在提升用户购物体验并促进实体经济的发展。

在需求分析阶段，项目详细分析了系统的功能需求、性能需求、安全需求和用户体验需求，并绘制了顶层用例图和系统流程图。系统架构设计采用分层架构，包括小程序端、视图层、业务层和数据层，各层次独立且相互协作，增强了系统的可维护性和可扩展性。功能模块设计覆盖用户、商家和管理员三大角色，提供了商品管理、订单管理、同城购物、虚拟试穿、实时客服聊天、直播等丰富的功能。

关键算法和模型部分重点介绍了协同过滤算法和基于 ResNet 的图像检索推荐算法，用于个性化商品推荐。同时，项目集成了 VisionKit 实现虚拟试鞋功能和改进的 VTON 模型实现虚拟试衣功能，通过 AR 和 AI 技术大幅提升用户的购物体验。

在界面设计和项目实现方面，智慧商城小程序注重用户界面的美观性和操作的便捷性，采用 Spring Boot 和 Flask 分别处理主要业务逻辑和虚拟试穿功能。系统测试部分进行了详细的功能测试和性能测试，确保各模块正常运行和系统高效性能。

本项目的创新点在于集成 AR 虚拟试鞋和 AI 虚拟试衣功能，为用户提供逼真的试穿体验；使用协同过滤和基于 ResNet 的图像检索推荐算法，提供个性化商品推荐；实现同城购物功能，支持快递配送和到店自取，促进实体经济发展；以及利用 WebSocket 实现用户与客服之间的双向沟通。实现直播功能，提高用户与商家互动的便利性。通过本项目的实施，不仅提升了用户的购物体验，也为商家提供了更加多样化的展示和销售方式，具有较高的商业应用价值和推广前景。

关键词：协同过滤算法；ResNet；高并发；AR；AI；直播；WebSocket

Smart Mall

Abstract: The Smart Mall Mini Program is an e-commerce platform developed based on the WeChat Mini Program framework, aiming to provide users with a convenient and efficient shopping experience. The project's development is driven by the rapid growth of e-commerce and the widespread adoption of smartphones, leading to increasing demand for mobile shopping solutions. However, existing e-commerce platforms are lacking in user experience, personalized recommendations, and virtual try-on capabilities. To address these shortcomings, this project integrates cutting-edge technologies such as AR, AI algorithms, and high-concurrency processing to enhance user shopping experiences and stimulate physical economy growth.

During the requirements analysis phase, the project thoroughly examined functional requirements, performance criteria, security needs, and user experience expectations, accompanied by top-level use case diagrams and system flowcharts. The system architecture follows a layered design encompassing the mini-program frontend, presentation layer, business logic layer, and data access layer, ensuring independence and collaboration among layers to enhance maintainability and scalability. Functional module design caters to users, merchants, and administrators, offering diverse features including product management, order processing, local shopping, virtual try-ons, real-time customer service chat, and live streaming.

Key algorithms and models focus on collaborative filtering and ResNet-based image retrieval for personalized product recommendations. Additionally, the integration of VisionKit enables virtual shoe try-ons, and the enhanced VTON model facilitates virtual clothing try-ons,

significantly elevating the shopping experience through AR and AI technologies.

In terms of interface design and implementation, the Smart Mall Mini Program emphasizes aesthetic user interfaces and intuitive interactions. Spring Boot and Flask are employed to handle core business logic and virtual try-on functionalities respectively. Rigorous functional and performance testing ensures the seamless operation and efficient performance of all modules.

The project's innovations lie in its integration of AR virtual try-ons and AI virtual clothing try-ons, providing realistic fitting experiences. It also utilizes collaborative filtering and ResNet-based image retrieval for personalized product recommendations, supports local shopping with delivery and in-store pickup options, fosters physical economy development, and enhances user engagement through real-time customer service chat and live streaming functionalities. By implementing this project, it not only enhances the shopping experience for users but also provides merchants with diversified display and sales methods, offering substantial commercial value and promising promotional prospects.

Key words : UBCF ; ResNet; High Concurrency; AR; AI; Live Streaming; Realtime Customer Service

目录

前言	1
第 1 章 绪论	1
1.1 项目背景	1
1.2 研究意义	1
1.3 发展动态	2
1.3.1 国内外发展状况	2
1.3.2 前沿技术	2
1.4 现有微信小程序电商平台的不足	2
1.5 本项目的创新点和目标	3
1.6 项目分工	4
第 2 章 需求分析	5
2.1 可行性分析	5
2.2 系统需求分析	6
2.2.1 功能需求	6
2.2.2 性能需求	8
2.2.3 安全需求	8
2.2.4 用户体验需求	9
2.3 顶层用例图	10
2.4 系统流程图	11
第 3 章 数据库设计	12
3.1 数据库表的设计	12
3.2 E-R 图	17
3.3 数据流图	17
第 4 章 系统架构与功能实现	19
4.1 系统架构设计	19
4.2 功能模块设计	20
4.3 系统主要功能实现	22

4.3.1	商品购买	22
4.3.2	阿里云 OSS	23
4.3.3	地址管理	24
4.3.4	购物车管理	25
4.3.5	评论管理	26
4.3.6	收藏商品管理	27
4.3.7	商品管理	29
4.3.8	订单管理	31
4.4	协同过滤算法	36
4.5	基于 ResNet 的图像检索推荐	39
第 5 章	项目实施与使用	51
5.1	项目实施	51
5.2	系统使用手册	52
5.3	项目效果	54
第 6 章	系统测试	62
6.1	测试目的	62
6.2	测试环境	62
6.3	功能测试	62
6.4	性能测试	69
6.5	总结	69
第 7 章	总结	70
7.1	克服的困难	70
7.2	升级演进	71
7.3	商业推广	71
7.4	感悟	72

前言

随着电子商务的迅猛发展，智慧商城小程序应运而生，旨在通过微信小程序平台为用户提供便捷的购物体验。项目通过集成创新技术如智能推荐、AR 试鞋、AI 换衣、实时客服聊天、直播模块以及同城购物功能，不仅提升了用户的互动体验和购物便利性，还促进了实体店经济和相关服务业的发展，进一步满足了用户多样化和个性化的消费需求。

第 1 章 绪论

1.1 项目背景

随着电子商务的迅猛发展，网络购物已经成为人们日常生活中的重要组成部分。智能手机的普及和移动互联网技术的进步，使得移动端购物平台得到了广泛的应用和推广。微信小程序作为一种轻量级的应用形态，凭借其无需安装、使用便捷、覆盖广泛等优势，逐渐成为电商平台的重要载体。智慧商城小程序应运而生，旨在为用户提供更加便捷和个性化的购物体验，同时为商家提供高效的商品管理和营销工具。本项目基于微信小程序平台，利用现代信息技术和人工智能算法，实现了多种创新功能，满足用户和商家的多样化需求。

1.2 研究意义

智慧商城小程序的研究和实现具有重要的理论和实际意义。在理论上，它丰富了电子商务系统的功能和架构设计，为未来电商系统的开发提供了新的思路 and 参考。在实际应用中，它为用户提供了更便捷、更智能的购物方式，提高了用户的购物体验，增强了用户粘性。同时，为商家提供了高效的商品管理和营销工具，促进了商家销售额的增长，具有显著的经济效益。同城购物功能的实现，不仅能够刺激本地消费，增加实体店的客流量，还可以带动网约车和外卖行业的发展，创造更多的就业机会，具有重要的社会效益。

1.3 发展动态

1.3.1 国内外发展状况

国内外电子商务平台在智能化和个性化服务方面均取得了一定的进展。例如，国内的淘宝、京东等平台已经开始应用大数据和人工智能技术提升用户体验；国外的 Amazon 也在不断优化其推荐算法和用户界面设计。同时，同城购物和本地化服务也逐渐成为电商平台的发展方向。此外，直播带货也成为电商平台新的增长点，通过直播互动增加用户粘性和购物体验。

1.3.2 前沿技术

1. **人工智能技术**：如机器学习、深度学习在推荐系统中的应用。
2. **增强现实（AR）技术**：在虚拟试衣、试鞋等场景中的应用。
3. **大数据技术**：用于用户行为分析和个性化推荐。
4. **本地化服务技术**：用于支持同城购物、快递配送和到店自取等功能。
5. **直播技术**：通过实时视频互动，实现直播带货功能，提高用户购物的参与感和即时性。

1.4 现有微信小程序电商平台的不足

目前，主流电商平台在用户体验和技术应用方面存在一些不足之处：

1. **个性化推荐不足**：传统的推荐系统主要依赖于用户的历史购买记录，推荐的准确性和实时性不足。
2. **缺乏互动性**：用户在购物过程中缺乏互动体验，如无法进行虚拟试衣、试鞋等操作。
3. **用户体验不佳**：复杂的操作流程和繁琐的界面设计，影响了用户的购物体验。
4. **同城购物支持不足**：现有平台对本地商品的支持和推广力度不够，用户难以找到本地优质商品。

-
5. **商家管理不便：**商家在商品管理、订单处理和用户互动等方面的工具和支持不足。
 6. **直播功能有限：**现有平台的直播功能无法充分满足商家和用户的互动需求，直播带货效果有限。

1.5 本项目的创新点和目标

创新点：

1. **虚拟试衣、试鞋功能：**利用增强现实（AR）技术，实现虚拟试衣、试鞋功能，提升用户购物体验。
2. **商品推荐算法：**基于协同过滤算法和 ResNet 图像检索算法，为用户提供精准的商品推荐。
3. **客服聊天：**实现使用 WebSocket 和客服聊天功能，为用户提供实时、高效的客服服务体验。
4. **同城购物功能：**支持用户浏览同城商品，选择快递配送或到店自取，满足用户多样化需求，并促进实体店经济、网约车和外卖行业的发展。
5. **高效的后台管理系统：**为商家提供商品管理、订单管理、直播等功能，提高商家运营效率。
6. **直播带货功能：**商家可以通过直播实时展示和销售商品，用户可以通过直播互动即时购买，提高购物体验和商家销售额。
7. **性能优化：**为提高系统的响应速度和并发能力，后端引入了 Redis 和 RabbitMQ 来进行缓存和消息队列处理。在推荐系统中，使用 Redis 缓存推荐结果，以减少相似度的重复计算，提升系统响应速度。在支付系统中，使用 RabbitMQ 进行异步下单处理。通过将订单请求放入消息队列，异步处理订单生成和支付操作，提高系统的并发处理能力，降低高并发场景下的系统负载。

目标：

1. 提供一个功能全面、操作简便的移动端购物平台，提升用户的购物体验。

-
2. 为商家提供高效的商品管理和营销工具，促进销售额增长。
 3. 利用先进的技术手段，实现个性化推荐、虚拟试衣等创新功能，增强平台竞争力。
 4. 推动同城购物，支持本地经济发展，带动相关行业如网约车和外卖的增长。
 5. 实现直播带货功能，提升用户购物的即时性和互动性，增加用户粘性。

1.6 项目分工

在智慧商城小程序的开发过程中，项目团队由两名同学组成。为了确保项目的高效进行和任务的合理分配，根据各自的技术特长和兴趣，进行了以下任务分解：

计算 214 徐祥禹主要负责：

1. **需求分析：**明确系统的功能需求和非功能需求，绘制顶层用例图、系统流程图、时序图、功能模块图并撰写报告。
2. **UI 设计：**确保各页面和功能模块的用户体验良好，设计并优化用户界面。
3. **前端开发：**使用微信小程序框架进行小程序的页面设计与开发。实现各个功能模块的视图展示，包括商品展示页、商品详情页、购物车、订单页面、个人中心等。实现用户交互功能，如用户登录注册、商品筛选与排序、购物车操作、订单操作等。整合 AR 试鞋和 AI 换衣功能，确保前端的用户体验。
4. **算法设计与优化：**实现关键算法如虚拟试衣算法，并进行优化。使用 Flask 框架处理虚拟试穿功能，确保 AR 试鞋和 AI 换衣功能的实现。
5. **同城购物模块：**设计并实现同城购物功能，包括商品展示、快递配送和到店自取选项。
6. **实时客服聊天：**WebSocket 实现实时客服聊天功能。
7. **功能与性能测试：**设计并实施功能测试和性能测试，确保各功能模块的正常运行和系统的高效性能。

计算 213 刘洋主要负责：

1. **系统架构设计：**提出系统的总体架构设计，绘制系统架构图并撰写报告。
2. **数据库设计与实现：**使用 MyBatis 进行数据库操作，设计并实现数据库结构和

主要数据表。确保数据的增删改查功能，优化数据库性能。使用 MySQL 存储核心数据，Redis 进行数据缓存。绘制 E-R 图、数据流图。

3. **后端开发：**使用 Spring Boot 框架进行主要业务逻辑的实现，包括用户管理、商品管理、订单管理等。实现与前端的 API 接口，保证数据传输的稳定与安全。
4. **算法设计与优化：**实现关键的推荐算法。实现协同过滤算法，根据用户的订单信息计算相似用户并推荐商品。使用基于 ResNet 的图像检索推荐算法，提取用户浏览过的商品图片特征，推荐视觉上相似的商品。
5. **直播模块：**实现腾讯云服务的直播功能，包括推流地址和视频接收的实现。
6. **系统部署与配置：**负责项目的部署与配置，确保系统在不同环境中的正常运行。编写系统安装与配置指南、系统使用手册及各类用户操作说明。

第 2 章 需求分析

进行需求分析是系统开发中不可或缺的环节，它有助于明确项目目标和范围，识别用户的实际需求和期望，制定合理的开发计划，降低开发风险。通过需求分析，项目团队能够确保所有参与者对项目的期望一致，深入了解用户痛点和需求，从而开发出真正满足用户需求的产品。这一步骤还为后续开发提供了基础数据和依据，有助于识别和预防潜在风险，确保项目顺利进行并提高成功率。

2.1 可行性分析

本文从技术、经济、市场和社会四个方面进行全面可行性评估。

在技术可行性方面，微信小程序作为一种轻量级应用，依托于微信庞大的用户基础和成熟的技术生态，具备良好的技术支持。小程序开发技术门槛较低，开发工具和资源丰富，开发团队具备相应的技术能力。此外，利用先进的人工智能算法和增强现实技术，能够实现个性化推荐和虚拟试衣等功能，具备较高的技术可行性。

从经济角度来看，开发智慧商城小程序的成本主要包括开发成本、运营维护成本和市场推广成本。与传统的电商平台相比，小程序的开发和维护成本相对较

低，且用户无需下载安装，降低了推广成本。通过平台的运营和商品销售收入，能够有效覆盖开发和运营成本，实现经济上的可行性。

市场方面，随着移动互联网的普及和用户购物习惯的改变，移动端购物已成为主流趋势。微信作为国内最大的社交平台，拥有庞大的用户群体，为智慧商城小程序提供了广阔的市场基础。通过个性化推荐、虚拟试衣、同城购物等创新功能，能够满足用户多样化的需求，具有较高的市场可行性。

从社会效益来看，智慧商城小程序的推出能够提升用户购物体验，促进本地经济发展，带动相关行业如网约车和外卖行业的增长。同时，项目的实施能够为商家提供更高效率的运营工具，提升商家的经济效益和市场竞争力，具有较高的社会可行性。

2.2 系统需求分析

2.2.1 功能需求

1. 用户功能

(1) 用户注册和登录

用户可以通过微信授权登录、账号密码登录。

提供退出登录和用户注册功能。

(2) 个人中心

修改密码、姓名等个人信息。

管理收货地址，包括添加、删除和修改地址。

查看和管理订单，包括全部订单、未支付订单、待发货订单和待收货订单。

查看收藏的商品，取消收藏。

(3) 商品浏览和购买

浏览首页的商品帖子。

根据大类和小类筛选商品。

根据上架时间、价格、销量等排序商品。

通过价格区间筛选商品。

浏览商品详情，查看商品规格。

添加商品到购物车。

选择收货地址进行购买。

进行支付。

(4) 商品评价

对已收货的商品进行评分（1-5）和文字评价。

对评价内容进行文本检验，确保无不合法信息。

(5) 商品推荐

根据订单信息使用协同过滤算法推荐相似用户购买过的商品。

利用 ResNet 神经网络对用户浏览过的商品图片进行特征提取，推荐视觉上相似的商品。

(6) 虚拟试穿

提供虚拟试鞋功能。

使用 VOfitFusion 模型实现虚拟试衣，用户上传照片后展示穿上当前商品的效果。

(7) 文件管理

上传和加载文件。

(8) 优惠券

管理和使用优惠券。

(9) 同城购物

浏览同城商品，选择快递配送或到店自取。

(10) 直播功能

观看商家的直播。

2. 商家功能需求

(1) 商品管理

管理商品信息，包括上传、修改、删除商品。

上传商品图片时进行检测。

(2) 订单管理

包括查看订单详情、处理发货等。

(3) 直播管理

发起和管理直播活动。

3. 管理员功能需求

(1) 用户管理

管理用户信息。

(2) 商家管理

管理商家信息。

(3) 首页管理

管理首页的商品帖子，包括审核和删除。

2.2.2 性能需求

1. 响应时间要求：

用户在浏览商品、下单、支付等操作时，系统应保证响应时间在 1 秒以内，以提供良好的用户体验。

2. 并发用户数：

系统应支持高并发访问，尤其是在促销活动或特定时间段内的用户访问高峰期，能够稳定处理大量请求。

3. 吞吐量：

系统应能处理每秒数百甚至数千次的请求，以确保在高流量情况下的稳定性和可靠性。

4. 资源利用率：

合理利用服务器资源，通过优化数据库查询、缓存机制和异步处理等手段，降低系统负载，提高系统整体效率。

2.2.3 安全需求

1. 用户数据安全：

用户密码和个人信息应使用加密存储，确保数据在传输和存储过程中的安全性。

使用 HTTPS 协议保护用户在网站和小程序间的数据传输安全。

2. 访问控制：

用户和管理员应有明确的访问权限控制，保证只有授权人员能够访问相应的功能模块和数据。

对于敏感操作（如支付、订单管理等），应实现双重验证或其他安全措施以防止未授权访问。

3. 数据备份与恢复：

定期对重要数据进行备份，确保在系统发生故障或数据丢失时能够快速恢复服务并保持数据完整性。

4. 代码安全性：

确保后端代码和数据库查询免受 SQL 注入、XSS 攻击等安全漏洞的影响。

对上传的文件进行检测和过滤，防止恶意文件上传和执行。

2.2.4 用户体验需求

1. 界面友好性：

设计简洁直观的用户界面，使用户能够快速找到所需功能和信息，减少学习成本。保持页面布局清晰、操作简单，提供一致的用户交互体验。

2. 快速响应：

系统应保证快速的响应速度和流畅的操作体验，避免长时间的加载和等待，特别是在移动网络环境下。

3. 个性化推荐：

基于用户的历史行为和偏好，提供个性化的商品推荐，增强用户对推荐内容的认可度和购买欲望。

利用协同过滤算法分析用户的订单信息，推荐相似用户购买过的商品。

使用 ResNet 神经网络对用户浏览过的商品图片进行特征提取，推荐视觉上相似的商品。

4. 虚拟试穿功能：

提供虚拟试鞋功能，用户可以在购买鞋子前通过 AR 技术进行虚拟试穿，增强购物体验。

使用 VOfitFusion 模型实现虚拟试衣功能，用户上传照片后可以看到自己穿上当前商品（如衣服、裤子）的效果，提升购物的趣味性和便利性。

5. 同城购物：

提供同城购物功能，用户可以浏览同城的商品，并选择快递配送或到店自取。

增加实体店经济，用户到店购物可以选择打车服务，促进网约车发展，或者选择外卖到家服务，促进骑手发展。

6. 实时反馈和客服聊天：

提供即时的操作反馈，例如订单状态变更、支付结果确认等，让用户能够及时了解操作的结果和状态。

7. 直播功能：

提供直播功能，用户可以观看商家的直播，了解商品详情和促销信息，增强互动性和购物体验。

2.3 顶层用例图

系统的顶层用例图展示了智慧商城的主要功能模块和用户角色的交互关系。该用例图包括三个主要角色：普通用户、商家和管理员。普通用户能够完成注册、登录、浏览商品、加入购物车、下单购买、商品评价、收藏商品、管理地址和支付等功能。商家则负责管理商品信息、处理订单并发起直播。管理员可以管理用户信息、商家信息以及首页帖子。通过顶层用例图，可以直观地看到各角色与系统功能之间的关系，帮助理清系统的主要需求和功能分布，为详细设计和开发提供指导。

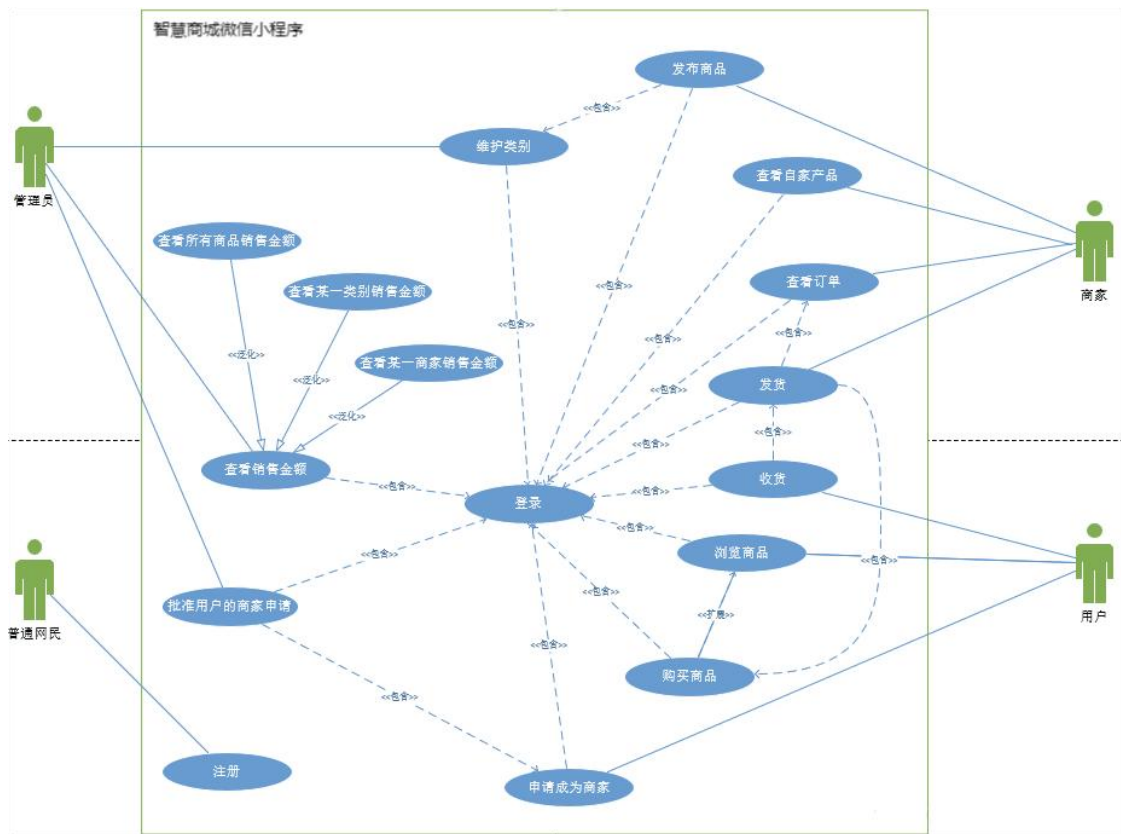


图 2-1 顶层用例图

2.4 系统流程图

系统流程图展示了智慧商城中用户和商家的主要业务流程。流程图涵盖了用户从注册、登录开始，浏览商品、选择商品规格、加入购物车、下单、选择地址、支付，再到订单生成、发货、收货和评价的全过程。同时，流程图还展示了商家管理商品信息、处理订单和发起直播的操作。每个流程节点细致地展现了数据如何在用户、前端界面、后台服务和数据库之间流转，确保系统能够顺利、高效地完成各项操作。通过系统流程图，能够清晰地理解各业务流程的逻辑顺序和数据流动，为系统开发和优化提供依据。

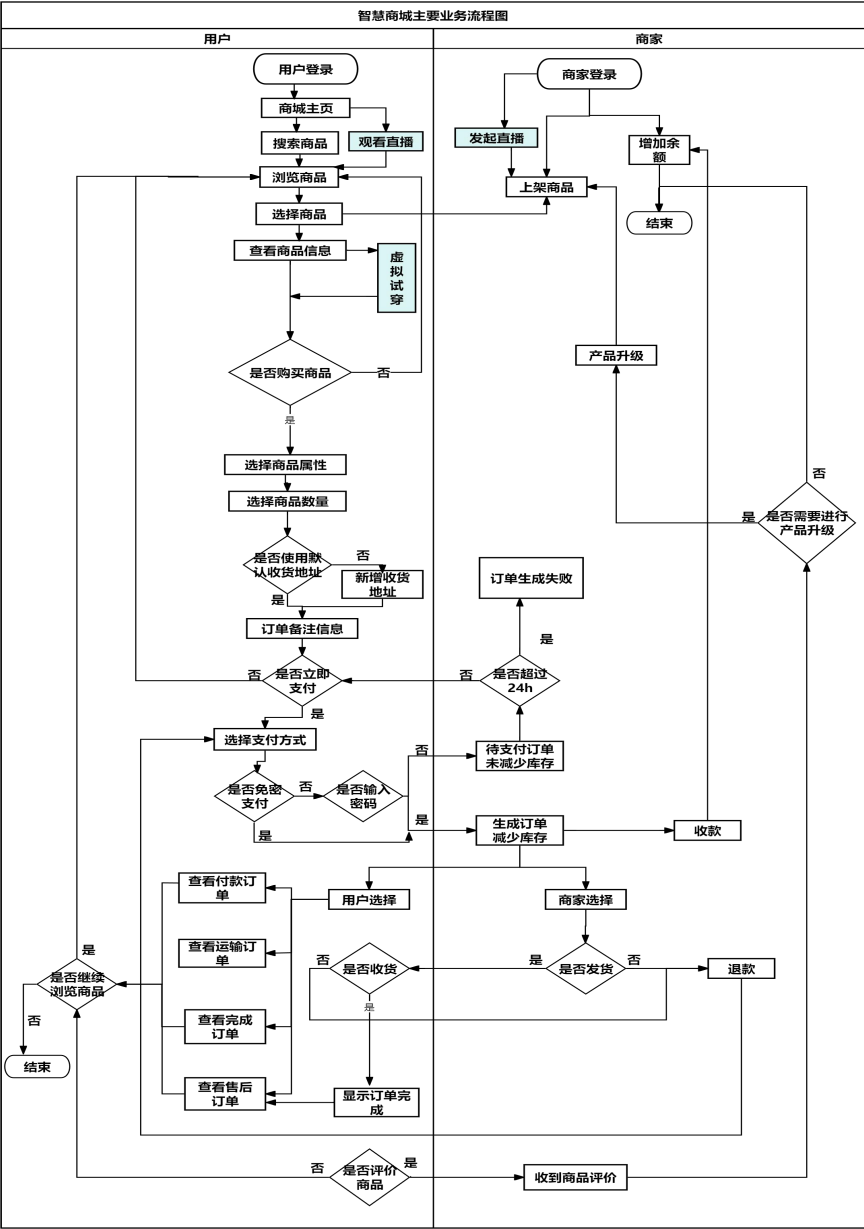


图 2-2 系统流程图

第 3 章 数据库设计

3.1 数据库表的设计

该数据库用于支持智慧商城小程序的运行，涵盖了用户、商品、订单等核心数据。数据库表协同工作，为智慧商城小程序提供了全面的数据支持，确保用户可以方便地进行商品浏览、购买、评价和收藏等操作。数据库包含以下主要表：

表 3-1address

字段	类型	主键	外键	允许空值	备注
id	int	主键		否	id
userid	int			否	用户 id
linkname	varchar(255)			是	收货人姓名
linkaddress	varchar(255)			是	收货地址
linkphone	varchar(11)			是	收货手机号

表 3-2 cart

字段	类型	主键	外键	允许空值	备注
id	int	主键		否	购物车 id
userid	int			否	用户 id
goodid	int			否	商品 id
standardid	int			否	规格 id

表 3-3 cate1

字段	类型	主键	外键	允许空值	备注
id	int	主键		否	大类 id
name	varchar(255)			否	大类名称

表 3-4 cate2

字段	类型	主键	外键	允许空值	备注
id	int	主键		否	小类 id
name	varchar(255)			否	小类名称
cate2img	varchar(255)			是	为了在分类里选择 图片+名称
cateidl	int			否	

表 3-5 comment

字段	类型	主键	外键	允许空值	备注
id	int	主键			商品评价 表
userid	int				
goodid	int				
orderid	int				
star	int				评分等级 1-5 颗星
description	text				评价描述

表 3-6 favorite

字段	类型	主键	外键	允许空值	备注
id	int	主键		否	id
goodid	int			否	收藏的商 品 id
userid	int			否	用户 id
lovetime	datetime			是	

表 3-7 good

字段	类型	主键	外键	允许空值	备注
id	bigint	主键			
cateid1	int			否	所属大类
cateid2	int			否	所属小类
name	varchar(255)			是	商品名
description	varchar(255)			是	商品描述

imgs	varchar(255)	是	图片路径
createtime	datetime	是	创建时间
salenum	int	是	总销售量
salemoney	int	是	总销售额
detail	varchar(255)	是	详情图片 商品 各角度的图片
shopid	int	是	商品所属店铺
model	varchar(255)	是	3D 模型位置

表 3-8 order

字段	类型	主键	外键	允许空值	备注
id	int	主键		否	订单 id
orderno	varchar(255)			否	订单编号
userid	int			否	用户 id
goodid	int			否	商品 id
standardid	int			否	规格 id
createtime	datetime			是	订单创建时间
status	varchar(255)			是	订单状态 未支付 已支付 已发货 已收货
linkname	varchar(255)			是	收货人姓名
linkaddress	varchar(255)			是	收货地址
linkphone	varchar(11)			是	收货手机号
addressid	int			否	地址信息 id

表 3-9 standard

字段	类型	主键	外键	允许空值	备注
id	int	主键		否	规格 id
goodid	int			否	商品 id

price	float	是	价格
store	int	是	库存
value	varchar(255)	否	规格描述

表 3-10 user

字段	类型	主键	外键	允许空值	备注
id	int	主键		否	用户 id
name	varchar(255)			是	用户名字/商家店铺名称/管理员名称
phone	varchar(11)			是	手机号
password	varchar(255)			是	登录密码
description	varchar(255)			是	个性签名/店铺描述
role	varchar(50)			是	身份 消费者/商家/管理员 user/shop/admin
avatar	varchar(255)			是	头像

表 3-11 message

属性名	类型	主键	外键	空值	备注
sender	int			是	发送者 ID
receiver	int			是	接收者 ID
msg	varchar(1024)			是	消息内容
time	varchar(1024)			是	消息发送时间
flag	int			是	消息标志
id	int	是		否	消息 ID (自增)

3.2 E-R 图

智慧商城的 ER 图通过图形化展示系统中的主要实体及其相互关系，明确了如用户、商品、订单等实体及其属性，帮助开发者和设计者直观理解数据库结构。它清晰定义了各实体之间的一对多、多对多等关系，支持系统的功能实现，如用户下单、商品管理、订单处理等。同时，ER 图在数据库设计过程中起到规范作用，确保数据的完整性和一致性，方便后续的数据库实现和优化。

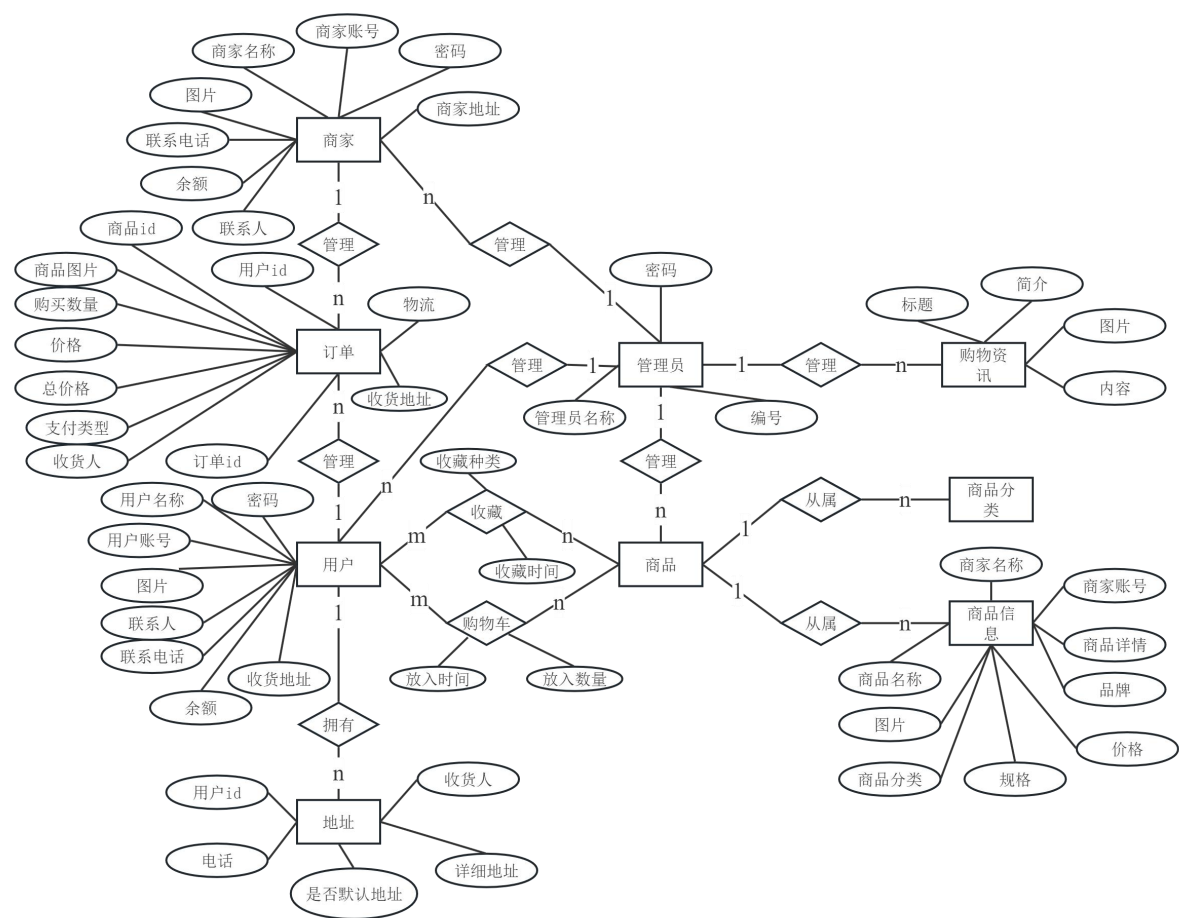


图 3-1 E-R 图

3.3 数据流图

智慧商城的数据流图通过图形化表示展示了系统中各模块之间的数据流转过程。它详细描述了从用户发起请求到系统处理、数据库交互及最终响应的各个步骤，帮助理解系统的整体工作流程。数据流图有效地展示了数据在不同功能模块之间的流动路径，如用户登录、商品浏览、购物车管理、订单处理、支付、评

价等，确保各环节紧密衔接，提高系统的设计和开发效率。

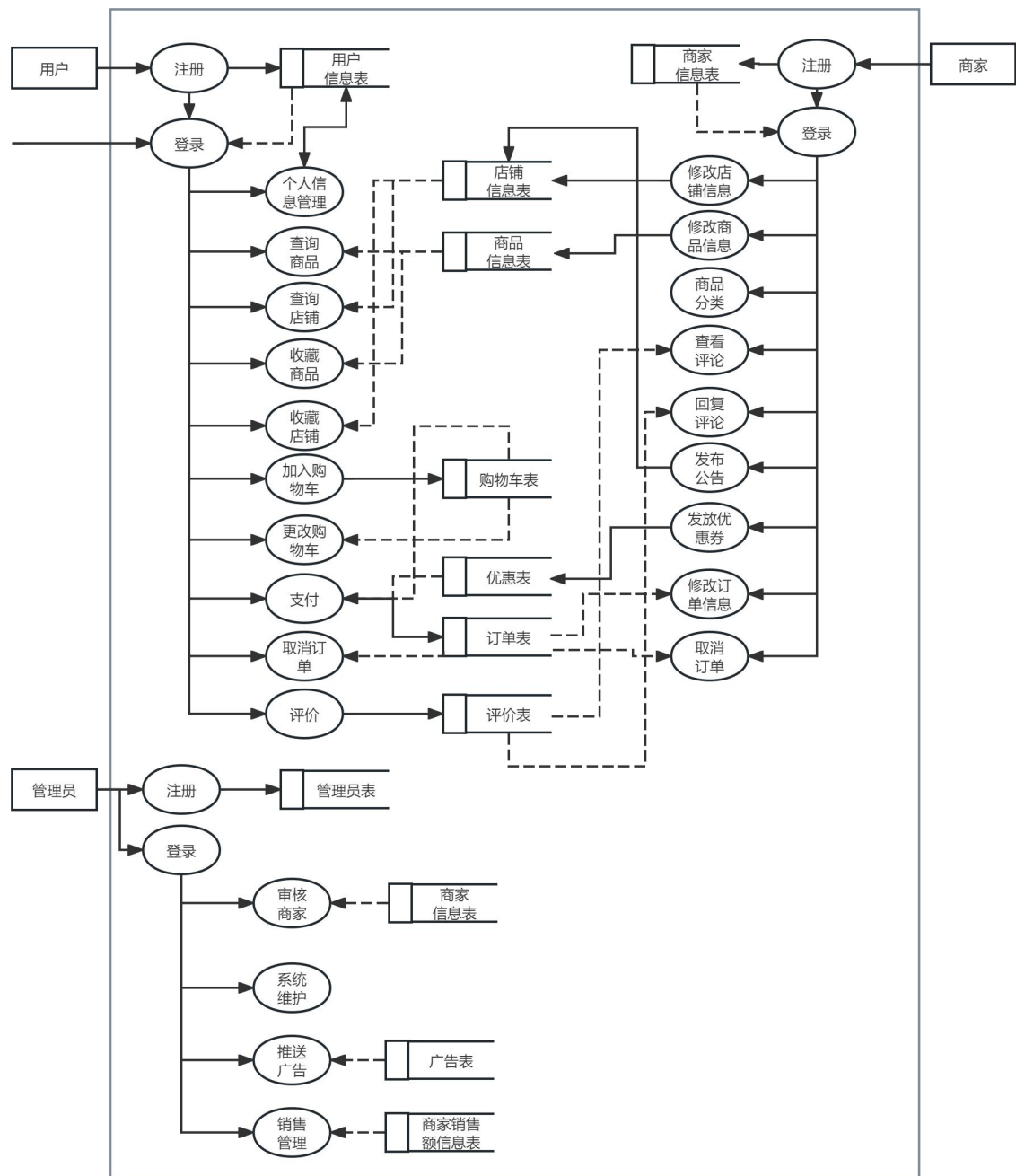


图 3-2 数据流图

第 4 章 系统架构与功能实现

4.1 系统架构设计

智慧商城系统总体架构采用前后端分离的设计原则，主要包括以下几个部分：

小程序端使用微信小程序框架，负责与用户进行交互。它主要处理用户输入，将用户请求发送到后端 API，并将后端返回的数据进行展示。微信小程序框架提供了良好的用户体验，简化了前端开发，提高了开发效率。

视图层基于微信小程序前端框架，实现各个功能模块的视图展示。通过使用 WXML、WXSS 和 JavaScript 等技术，视图层构建了丰富的用户界面，展示数据并处理用户操作，确保用户可以直观、便捷地与系统交互。

业务层主要由 Spring Boot 和 Flask 框架组成，处理系统的核心业务逻辑。Spring Boot 负责实现用户管理、商品管理、订单管理、支付、评价、收藏、推荐、同城购物和直播等功能。Flask 则专注于处理虚拟试穿功能，利用 VisionKit 和 VOfitFusion 模型实现虚拟试鞋和试衣功能。业务层的设计确保了系统功能的完整性和高效性。

数据层使用 MyBatis 持久层框架负责数据库操作，进行数据的增删改查。MyBatis 提供了数据访问接口，简化了数据库操作，提高了数据处理效率，确保了系统数据的完整性和一致性。

数据库层包括 MySQL 和 Redis。MySQL 用于存储核心数据，包括用户信息、商品信息、订单信息、评价、收藏等。Redis 用于缓存数据，存储临时数据以提高系统响应速度，缓解数据库压力。两者的结合保证了数据存储的可靠性和系统性能的优化。

通过这种分层设计，智慧商城系统能够实现各个模块的独立开发和维护，提高系统的稳定性和扩展性，确保用户能够享受到流畅的使用体验。系统架构图如下。

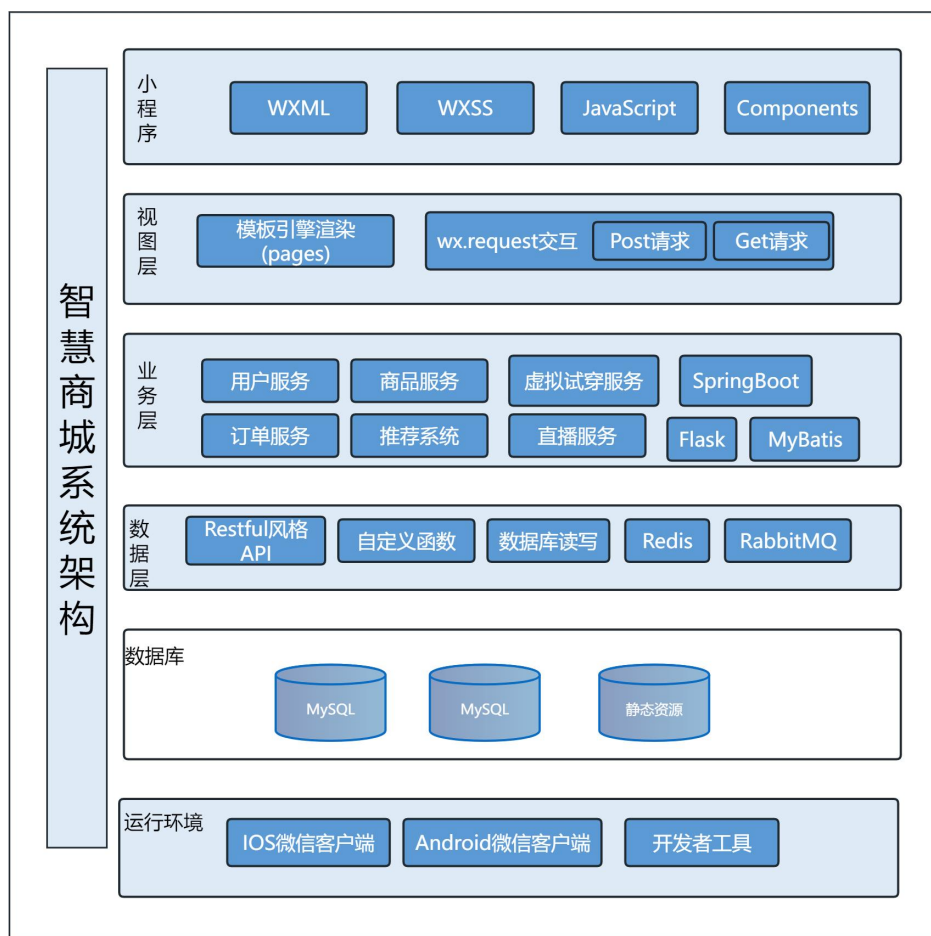


图 4-1 系统架构图

4.2 功能模块设计

用户功能设计：

智慧商城系统为用户提供了丰富且便捷的功能，以提升整体购物体验。用户可以通过微信授权登录快速进入系统，进行注册或退出登录。首页展示用户上传的商品帖子，用户可以浏览和筛选商品，按照上架时间、价格、销量等进行排序，也可以根据大类或小类进行筛选。用户可以管理收货地址，添加、删除和修改地址信息。购买过程中，用户可以查看商品详情，选择规格，选择配送地址并提交订单，使用虚拟支付完成交易。

系统还提供了个人中心功能，用户可以修改密码、姓名等信息；订单页面展示了所有订单，包括未支付、待发货和待收货的订单。用户可以对已收货商品进行评价，查看和管理收藏的商品，并将商品加入购物车。此外，系统基于协同过

滤算法和 ResNet 图像检索算法，为用户提供商品推荐服务。用户还可以体验虚拟试鞋和试衣功能，前者通过 VisionKit 实现，后者通过 VOfitFusion 模型实现。

为增强用户体验，系统支持同城购物功能，用户可以浏览同城商品，选择快递配送或到店自取，促进实体店经济和本地配送服务的发展。系统还集成了 WebSocket，实现了客服聊天功能，用户可以随时与客服进行实时沟通，解决购物过程中的问题。

商家功能设计：

智慧商城系统为商家提供了全面的商品和订单管理功能，帮助商家高效运营。商家可以通过系统管理自己的商品信息，包括添加、删除和修改商品，确保商品信息的准确性和更新。订单管理功能使商家能够查看和处理用户的订单，管理订单的发货状态，保证及时交付商品。系统还支持商家发起直播，利用直播功能进行商品展示和推广，吸引更多的用户关注和购买。同时，商家可以查看用户对其商品的评价，了解用户反馈，进行改进和优化。通过这些功能，商家能够更加便捷地管理业务，提升用户满意度，促进销售增长。

管理员功能设计：

智慧商城系统中的管理员角色负责管理整个系统的运营和维护。管理员拥有全面的管理权限，包括用户信息管理、商家信息管理和首页帖子管理。用户信息管理模块允许管理员查看和管理所有用户的账户信息，确保用户信息的准确性和安全性。商家信息管理模块使管理员能够审核和管理商家的注册信息和商品发布信息，保证平台上商家的合法性和商品的合规性。首页帖子管理模块让管理员能够审核用户发布的商品帖子，删除不符合规定的内容，维护平台的内容质量。管理员还可以管理商品评价，对用户提交的商品评价进行审核和处理，确保评价内容的合法性和真实性。此外，管理员能够查看和处理所有订单，监督订单的状态和处理进度。

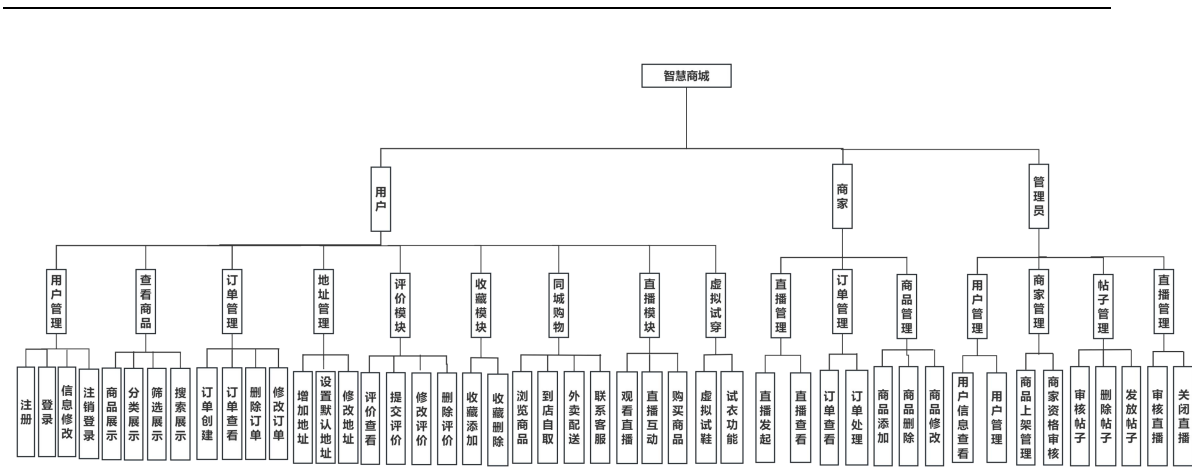


图 4-2 功能模块图

4.3 系统主要功能实现

4.3.1 商品购买

时序图展示了系统中各个组件或模块之间的交互过程及其时间顺序，通过垂直时间轴上的消息传递来描述系统行为。它有助于理解系统在执行某一特定任务时各个部分的协作方式和调用顺序，从而发现潜在的逻辑问题和优化点。通过时序图，可以清晰地看到用户、前端、后端以及数据库等各部分在处理请求时的详细交互步骤，为开发和调试提供直观的参考。商品购买流程的时序图如下：

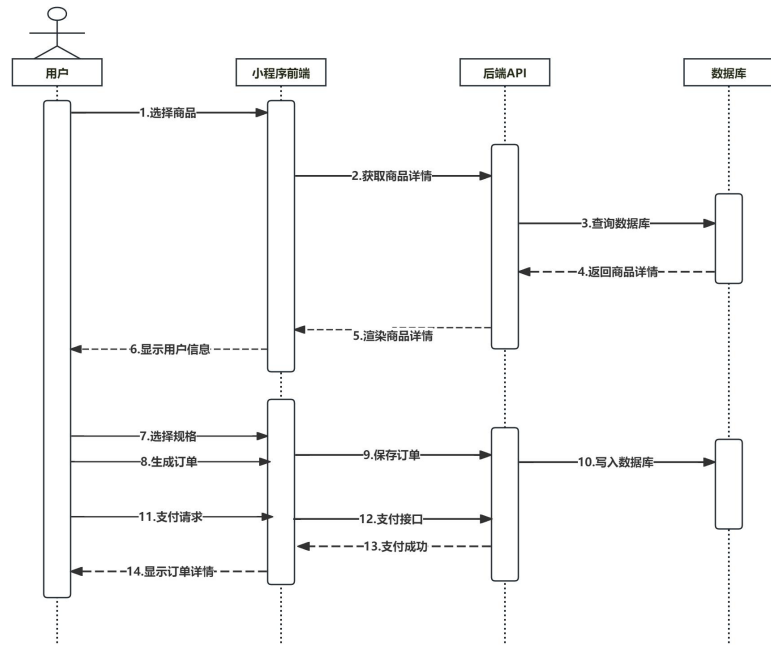


图 4-3 商品购买的时序图

4.3.2 阿里云 OSS

阿里云 OSS（对象存储服务）工具类，用于将图片上传到阿里云的 OSS 存储桶。通过 upload 方法，该类接收一个 MultipartFile 类型的文件，将其上传到指定的 OSS 存储桶中，并生成一个唯一的文件名以避免文件覆盖，最终返回上传文件的访问 URL。

```
package com.example.mall.utils;

import com.aliyun.oss.OSS;
import com.aliyun.oss.OSSClientBuilder;
import org.springframework.stereotype.Component;
import org.springframework.web.multipart.MultipartFile;
import java.io.*;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

/**
 * 阿里云 OSS 工具类
 */
@Component
public class AliOSSUtils {

    private String endpoint = "https://oss-cn-qingdao.aliyuncs.com";
    private String accessKeyId = "LTAI5t9EFCX38GSVmXaaMX2f";
    private String accessKeySecret = "TlxHA13MPccTUBaFT5r6DAk3SB2PZG";
    private String bucketName = "lyxxy";

    /**
     * 实现上传图片到 OSS
     */
    public String upload(MultipartFile multipartFile) throws IOException {
        // 获取上传的文件的输入流
        InputStream inputStream = multipartFile.getInputStream();

        // 避免文件覆盖
        String fileName = LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyy-MM-dd-HH-mm-ss")) + multipartFile.getOriginalFilename();

        // 上传文件到 OSS
        OSS ossClient = new OSSClientBuilder().build(endpoint, accessKeyId, accessKeySecret);
```

```

        ossClient.putObject(bucketName, fileName, inputStream);

        //文件访问路径
        String url = endpoint.split("///")[0] + "///" + bucketName + "." + endpoint.split("///")[1] + "/" + fileName;
        // 关闭 ossClient
        ossClient.shutdown();
        return url; // 把上传到 oss 的路径返回
    }
}

```

4.3.3 地址管理

地址管理的控制器 `AddressController`，用于处理用户地址的相关操作。包括获取某用户的所有地址、添加新地址、根据 ID 获取地址、更新地址以及删除地址。控制器通过 `AddressService` 服务类与数据库进行交互，并通过统一的 `Result` 类封装返回结果。各个方法分别对应 RESTful 风格的 GET、POST、PUT 和 DELETE 请求。

```

package com.example.mall.controller;

import com.example.mall.common.Result;
import com.example.mall.entity.Address;
import com.example.mall.service.AddressService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/address")
public class AddressController {

    @Autowired
    private AddressService addressService;

    @GetMapping("/getall/{userId}")
    public Result getUserAddresses(@PathVariable("userId") int userId) {

```

```
        List<Address> addresses = addressService.getUserAddresses(u
serId);

        return Result.success(addresses);
    }

    @PostMapping("/addAddress")
    public Result addAddress(@RequestBody Address address) {
        addressService.addAddress(address);
        return Result.success();
    }

    @GetMapping("/address/{id}")
    public Result getAddressById(@PathVariable("id") int id) {
        Address address = addressService.getAddressById(id);
        System.out.println(address);
        return Result.success(address);
    }

    @PutMapping("/updateAddress/{id}")
    public Result updateAddress(@PathVariable("id") int id, @RequestBod
y Address address) {
        address.setId(id);
        addressService.updateAddress(address);
        return Result.success();
    }

    @DeleteMapping("/deleteAddress/{id}")
    public Result deleteAddress(@PathVariable("id") int id) {
        addressService.deleteAddress(id);
        return Result.success();
    }
}
```

4.3.4 购物车管理

购物车管理的控制器 `CartController`，用于处理用户购物车的相关操作。包括添加商品到购物车、获取某用户的购物车项以及删除购物车项。

```
package com.example.mall.controller;

import com.example.mall.entity.CartItemDTO;
import com.example.mall.entity.Cart;
```

```
import com.example.mall.service.CartService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/cart")
public class CartController {

    @Autowired
    private CartService cartService;

    @PostMapping("/add")
    public ResponseEntity<String> addCart(@RequestBody Cart cart) {
        cartService.addCart(cart);
        return ResponseEntity.ok("Successfully added to cart");
    }

    @GetMapping("/user/{userid}")
    public ResponseEntity<List<CartItemDTO>> getCartItems(@PathVariable int
userid) {
        List<CartItemDTO> cartItems = cartService.getCartItemsByUserId(
userid);
        return ResponseEntity.ok(cartItems);
    }

    @DeleteMapping("/delete/{cartId}")
    public void deleteCartItems(@PathVariable int cartId) {
        cartService.deleteCartItemsBycartId(cartId);
    }
}
```

4.3.5 评论管理

评论管理的控制器 `CommentController`，用于处理与商品评论相关的操作。主要功能包括添加评论和根据商品 ID 获取评论列表。

```
package com.example.mall.controller;

import com.example.mall.entity.CartItemDTO;
```

```
import com.example.mall.entity.Cart;
import com.example.mall.service.CartService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/cart")
public class CartController {

    @Autowired
    private CartService cartService;

    @PostMapping("/add")
    public ResponseEntity<String> addCart(@RequestBody Cart cart) {
        cartService.addCart(cart);
        return ResponseEntity.ok("Successfully added to cart");
    }

    @GetMapping("/user/{userid}")
    public ResponseEntity<List<CartItemDTO>> getCartItems(@PathVariable int
userid) {
        List<CartItemDTO> cartItems = cartService.getCartItemsByUserId(
userid);
        return ResponseEntity.ok(cartItems);
    }

    @DeleteMapping("/delete/{cartId}")
    public void deleteCartItems(@PathVariable int cartId) {
        cartService.deleteCartItemsBycartId(cartId);
    }
}
```

4.3.6 收藏商品管理

FavoriteController 提供了管理用户商品收藏的功能，包括添加收藏、删除收藏、检查收藏是否存在，以及获取用户收藏的商品列表。通过相应的 RESTful API 接口，前端可以与后端进行交互，完成对收藏信息的增删查操作。

```
package com.example.mall.controller;
```

```
import com.example.mall.entity.Favorite;
import com.example.mall.entity.Good;
import com.example.mall.service.FavoriteService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.Date;
import java.util.List;

@RestController
@RequestMapping("/favorite")
public class FavoriteController {
    @Autowired
    private FavoriteService favoriteService;

    // 添加收藏
    @PostMapping("/add")
    public int addFavorite(@RequestBody Favorite favorite) {
        favorite.setLovetime(new Date()); // 设置为当前时间
        return favoriteService.addFavorite(favorite);
    }

    // 删除收藏
    @DeleteMapping("/remove/{goodid}/{userid}")
    public int removeFavorite(@PathVariable int goodid, @PathVariable int userid) {
        return favoriteService.removeFavorite(goodid, userid);
    }

    // 查询是否已经存在过当前用户当前商品的收藏信息
    @GetMapping("/checkExist/{goodid}/{userid}")
    public boolean checkExist(@PathVariable int goodid, @PathVariable int userid) {
        return favoriteService.checkExist(goodid, userid);
    }

    // 获取用户收藏的商品列表
    @GetMapping("/list/{userid}")
    public List<Good> getFavoriteGoods(@PathVariable int userid) {
        return favoriteService.findFavoritesByUserId(userid);
    }
}
```

在线工具 由 [O](#)

4.3.7 商品管理

GoodController 提供了管理商品的功能，包括根据 ID 查找商品、插入新商品、更新商品、删除商品、根据类别查找商品、过滤商品、查找所有商品、根据用户 ID 查找商品以及关键字搜索商品。通过这些 RESTful API 接口，前端可以与后端进行交互，完成对商品信息的各种操作，支持复杂的商品筛选和搜索功能。

```
package com.example.mall.controller;

import com.example.mall.common.Constants;
import com.example.mall.common.Result;
import com.example.mall.entity.Good;
import com.example.mall.service.GoodService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/goods")
public class GoodController {
    @Autowired
    private GoodService goodService;

    @GetMapping("/{id}")
    public Result findById(@PathVariable Long id) {
        Good good = goodService.findById(id);
        if (good == null) {
            // return ResponseEntity.notFound().build();
            return Result.error("商品不存在");
        }
        System.out.println(good);
        return Result.success(good);
    }
}
```

```

@PostMapping("/insert")
public void insert(@RequestBody Good good) {

    System.out.println(good);
    goodService.insert(good);
}

@PutMapping("/{id}")
public void update(@PathVariable Long id, @RequestBody Good good) {
    good.setId(id);
    goodService.update(good);
}

@DeleteMapping("/{id}")
public void delete(@PathVariable Long id) {
    goodService.delete(id);
}

@PostMapping(value = "/category" )
public Result findByCategoryIdWithMinPrice(@RequestParam Integer categoryId) {
    List<Map<String, Object>> goods = goodService.findByCategoryIdWithMinPrice(categoryId);
    return Result.success(goods);
}

/** 根据大类 小类查询商品 */
@PostMapping(value = "/getcategory/{cate1id}/{cate2id}")
public Result findByCateIds(@PathVariable Integer cate1id, @PathVariable Integer cate2id) {
    List<Map<String, Object>> goods = goodService.findByCateIds(cate1id, cate2id);
    System.out.println(goods);
    return Result.success(goods);
}

@GetMapping("/{cateid1}/{cateid2}")
public Result findByCateIdsWithOrderBy(@PathVariable Integer cateid1, @PathVariable Integer cateid2, @RequestParam(required = false, defaultValue = "createtime") String orderBy) {
    List<Map<String, Object>> goods = goodService.findByCateIdsWithMinPriceAndOrderBy(cateid1, cateid2, orderBy);
    return Result.success(goods);
}

```

```

    @PostMapping("/filter/{cate1id}/{cate2id}/{minPrice}/{maxPrice}")
    public Result filterGoods(@PathVariable Integer cate1id, @PathVariable
Integer cate2id,
                                @PathVariable Double minPrice, @PathVariable
Double maxPrice) {
        List<Map<String, Object>> filteredGoods = goodService.filterByPrice
Range(cate1id, cate2id, minPrice, maxPrice);
        return Result.success(filteredGoods);
    }

    @GetMapping("/findall")
    public Result findAll() {
        List<Good> goods = goodService.findAll();
        return Result.success(goods);
    }

    @GetMapping("/findallbyid")
    public Result findAllById(@RequestParam("userId") Long userId) {
        List<Good> goods = goodService.findAllById(userId);
        return Result.success(goods);
    }

    @PostMapping("/search")
    public Result searchGoods(@RequestBody Map<String, String> params) {
        String keyword = params.get("keyword");
        List<Map<String, Object>> goods = goodService.searchGoods(keyword);
        return Result.success(goods);
    }
}

```

4.3.8 订单管理

`OrderController` 提供了订单管理的功能，包括推荐商品、创建订单、确认订单、获取用户订单列表和订单详情。推荐功能结合了基于协同过滤和 ResNet 的推荐结果，创建订单时生成订单号并保存详细信息，确认订单时更新订单状态，获取用户订单列表时格式化时间戳以便展示，此外还可以根据订单 ID 获取订单

详细信息。通过这些 RESTful API 接口，前端可以与后端进行交互，完成订单的创建、查询和确认操作。

```
package com.example.mall.controller;

import com.example.mall.api.ResNet;
import com.example.mall.common.Result;
import com.example.mall.entity.Address;
import com.example.mall.entity.Good;
import com.example.mall.entity.Order;
import com.example.mall.service.AddressService;
import com.example.mall.service.GoodService;
import com.example.mall.service.OrderService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.text.SimpleDateFormat;
import java.time.LocalDateTime;
import java.util.*;
import java.util.stream.Collectors;

@RestController
@RequestMapping("/order")
public class OrderController {
    @Autowired
    private OrderService orderService;
    @Autowired
    private AddressService addressService;
    // @Autowired
    // private OrderService orderService;

    @Autowired
    private GoodService goodService;
    @GetMapping("/recommend/{userid}")
    public Result recommend(@PathVariable int userid) {
        // 获得推荐商品的 id
        List<Integer> recommendedGoodsIds = orderService.recommendGoods(userid);

        System.out.println(recommendedGoodsIds);

        List<Integer> recommendResnet= ResNet.getRecommendedGoods(userid);
        System.out.println("-----resnet-----");
        System.out.println(recommendResnet);
    }
}
```

```

        // 使用 Set 来合并并去重
        Set<Integer> mergedList = new HashSet<>();
        mergedList.addAll(recommendedGoodsIds);
        mergedList.addAll(recommendResnet);
        // 转换回 List 并排序（如果需要）
        List<Integer> mergedAndSorted = new ArrayList<>(mergedList);
        Collections.sort(mergedAndSorted);

        // 根据商品 id 获取商品详细信息
        List<Good> recommendedGoods = mergedAndSorted.stream()
            .map(id -> goodService.findByIdWithPrice(Long.valueOf(id)))
            .collect(Collectors.toList());

        return Result.success(recommendedGoods);
    }

    @PostMapping("/create")
    public Result createOrder(@RequestBody Order order) {
        // 生成订单号（可根据当前时间和一定规则生成）
        String orderNo = generateOrderNo();

        // 设置订单信息
        order.setOrderno(orderNo);
        // 获取当前时间
        Date date = new Date();
        order.setCreatetime(date);
        order.setStatus("待收货"); // 待支付状态
        // 获取地址信息
        Address address=addressService.getAddressById(order.getAddressid());
;
        order.setLinkaddress(address.getLinkaddress());
        order.setLinkname(address.getLinkname());
        order.setLinkphone(address.getLinkphone());
        // 保存订单
        boolean suc=orderService.createOrder(order);
        if(suc){

            System.out.println("-----创建订单-----");
            System.out.println(order);
            return Result.success("创建订单成功!");
        }
        else{
            return Result.error();
        }
    }
}

```

```

    @PostMapping("/confirm")
    public Result confirm(@RequestParam int orderId) {
        // 生成订单号（可根据当前时间和一定规则生成）
        Order order = orderService.getOrderById(orderId);
        order.setStatus("待评价");
        System.out.println(order);
        orderService.update(order);
        return Result.success();
    }

    // 获取用户订单列表
    @GetMapping("/list/{userid}")
    public Result getOrderList(@PathVariable int userid) {
        List<Order> orders = orderService.getOrderListByUserId(userid);
        // 格式化时间
        SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
        for (Order order : orders) {
            order.setFormattedTime(sdf.format(order.getCreatetime() ));
        }
        return Result.success(orders);
    }

    // 获取订单详情
    @GetMapping("/{id}")
    public Result getOrderById(@PathVariable int id) {
        Order order = orderService.getOrderById(id);
        if (order != null) {
            return Result.success( order);
        } else {
            return Result.error("订单不存在");
        }
    }

    // 可根据具体需求修改订单号生成规则
    private String generateOrderNo() {
        return UUID.randomUUID().toString().replace("-", "").substring(0, 2
0);
    }

}

```

4.3.9 商品管理

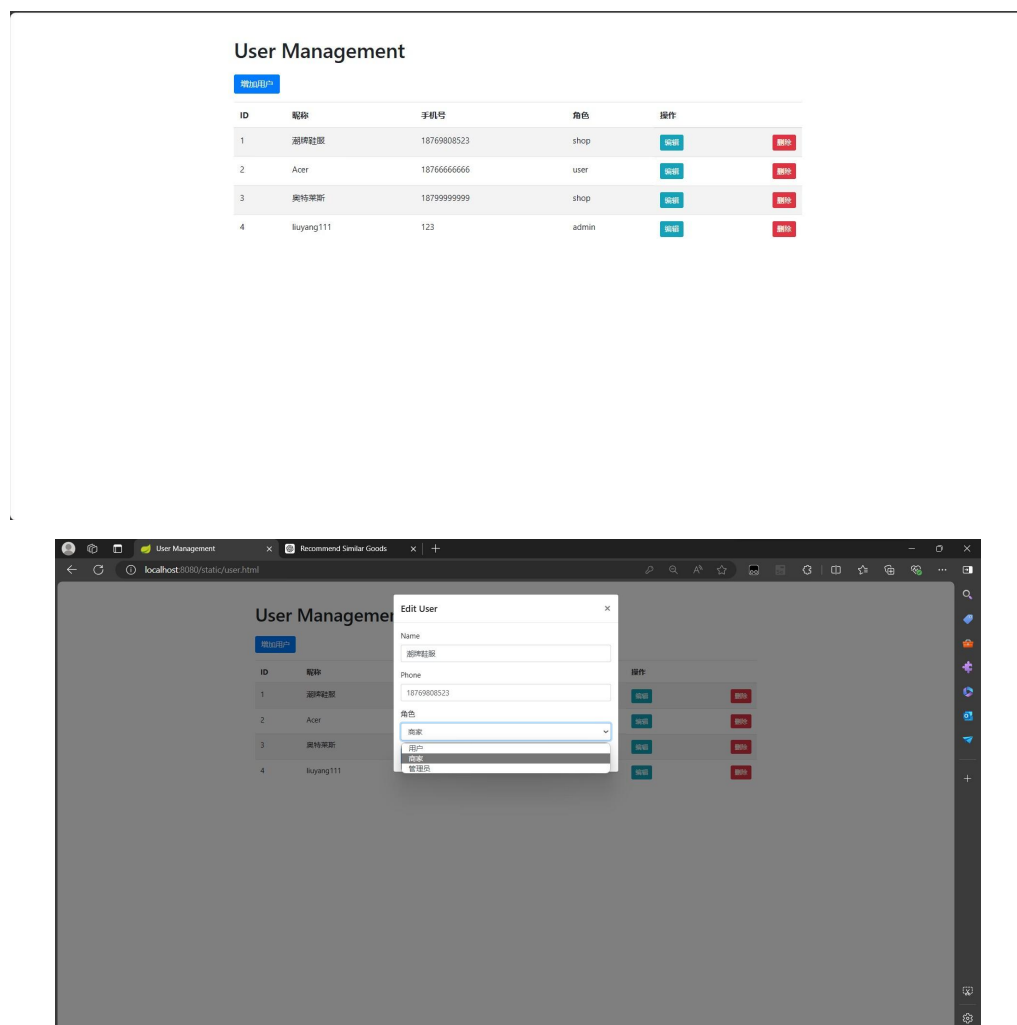
商家在后台管理系统中可以通过 web 页面与后端 API 的交互，实现商品的增删改查操作。

商家登录系统后，通过商品管理页面可以看到商品列表，点击“新增”按钮可以进入商品添加页面，填写相关信息后提交；点击“编辑”按钮可以修改商品信息，保存后更新数据库；点击“删除”按钮可以移除商品；在搜索框中输入关键词可以查询商品信息。后台 API 通过 RESTful 接口与前端进行数据交互，完成对数据库的增删改查操作。

The screenshot shows a web interface for editing a product. At the top, there's a header '商品管理' (Product Management) with a green '增加商品' (Add Product) button. Below it, a form titled '编辑商品' (Edit Product) is displayed. The form includes the following fields: 'NIKE AIR FORCE 1' (Product Name), 'Nike Air Force 1 07 经典复古风格 包边性保暖 低帮 板鞋 男款 纯白' (Product Description), '鞋类' (Category) with a dropdown arrow, '板鞋' (Brand) with a dropdown arrow, 'https://yxyy-oss-cn-qingdao.aliyuncs.com/kongjun1.jpg' (Image URL), '2024-05-05 08:09' (Creation Time), '100015' (Stock Quantity), '10' (Another field), 'https://yxyy-oss-cn-qingdao.aliyuncs.com/kongjun10.jpg' (Another Image URL), '1' (Another field), '3D模型位置' (3D Model Position), and '2' (Another field). At the bottom of the form are two buttons: '保存' (Save) in green and '删除' (Delete) in red.

4.3.10 用户管理

管理员登录后台管理系统后，可以通过用户管理页面查看所有用户的信息，点击“添加用户”按钮可以新增用户，填写信息并提交；点击用户列表中的“编辑”按钮可以修改用户信息并保存；点击“删除”按钮可以移除用户；使用搜索和过滤功能可以快速查找和筛选用户。此外，管理员可以为用户分配不同的角色和权限，通过前端与后端 API 的交互实现对数据库中用户信息的增删改查操作。



4.4 协同过滤算法

基于用户的协同过滤（User-Based Collaborative Filtering）是推荐系统中常用的一种算法，它基于用户之间的相似性来进行推荐。其基本思想是：如果两个用户在过去喜欢或者不喜欢的物品比较相似，那么他们在将来可能也会有相似的良好。

1. 计算用户相似度：对于每一对用户，计算其之间的相似度。

找到邻居用户：对于每个用户，找到与其相似度最高的一组邻居用户。通常会设定一个阈值或者选择固定数量的邻居用户。

2. 生成推荐列表：对于每个用户，根据其邻居用户的喜好情况，生成推荐列表。可以根据邻居用户的评分加权平均值或者其他方法来生成推荐列表。

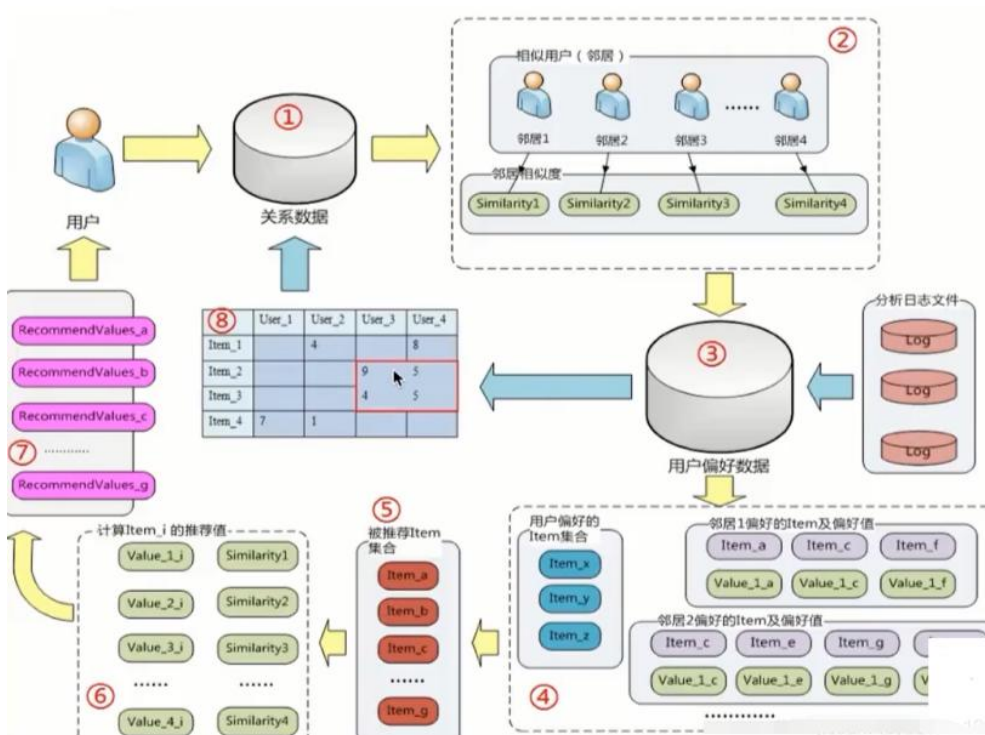


图 5-1 协同过滤原理图

(1) 找到与目标用户兴趣相似的用户集合

利用行为的相似度计算兴趣的相似度。给定用户 u 和用户 v ,

令 $N(u)$ 表示用户 u 曾经有过正反馈的物品集合

令 $N(v)$ 为用户 v 曾经有过正反馈的物品集合

可通过如下公式简单地计算 u 和 v 的兴趣相似度：

Jaccard 公式: $w_{uv} = \frac{|N(u) \cap N(v)|}{|N(u) \cup N(v)|}$

余弦相似度: $w_{uv} = \frac{|N(u) \cap N(v)|}{\sqrt{|N(u)| |N(v)|}}$

(2) 推荐商品

首先需要从矩阵中找出与目标用户 u 最相似的 K 个用户，用集合 $S(u, K)$ 表示，将 S 中用户喜欢的物品全部提取出来，并去除 u 已经喜欢的物品。对于每个候选物品 i ，用户 u 对它感兴趣的程度用如下公式计算：

$$p(u, i) = \sum_{v \in S(u, K) \cap N(i)} w_{uv} \times r_{vi}$$

其中 r_{vi} 表示用户 v 对 i 的喜欢程度，在一些需要用户给予评分的推荐系统

中，则要代入用户评分。

根据最后得到的 p ，推荐给用户 rank 前 N 的物品。

实现代码：

```
/****** 协同过滤算法推荐商品 ******/
public List<Long> recommendProductsBasedOnPurchaseHistory(Long userId) {
    // 步骤 1: 构建用户到商品的映射
    Map<Long, Set<Long>> userToProductsMap = getUserToProductsMap();
    ;
    // System.out.println("userToProductsMap"+userToProductsMap.toString());
    // 当前用户的购买商品集合
    Set<Long> currentUserProducts = userToProductsMap.getOrDefault(
        userId, Collections.emptySet());
    System.out.println("currentUserProducts"+currentUserProducts.toString());
    // 步骤 2: 计算用户间的相似度
    Map<Long, Double> userSimilarityScores = new HashMap<>();
    for (Map.Entry<Long, Set<Long>> entry : userToProductsMap.entrySet()) {
        if (!entry.getKey().equals(userId)) {
            Set<Long> otherUserProducts = entry.getValue();
            // 计算 Jaccard 相似度
            double similarityScore = calculateJaccardSimilarity(currentUserProducts, otherUserProducts);
            userSimilarityScores.put(entry.getKey(), similarityScore);
        }
    }
    // 步骤 3: 为目标用户找到最相似的用户
    List<Long> mostSimilarUserIds = userSimilarityScores.entrySet().stream()
        .sorted(Map.Entry.comparingByValue(Comparator.reverseOrder()))
        .map(Map.Entry::getKey)
        .limit(1) // 假设我们取相似度最高的 100 个用户
        .collect(Collectors.toList());
    // 步骤 4: 生成商品推荐
    Set<Long> recommendedProducts = new HashSet<>();
    //先加上自己买过的商品
```

```
        recommendedProducts.addAll(currentUserProducts);
//        System.out.println("目前包含的");
//        System.out.println(recommendedProducts);
        for (Long similarUserId : mostSimilarUserIds) {
            Set<Long> similarUserProducts = userToProductsMap.get(similarUserId);
            for (Long productId : similarUserProducts) {
                recommendedProducts.add(productId);
            }
        }
        return new ArrayList<>(recommendedProducts);
    }
}
```

4.5 基于 ResNet 的图像检索推荐

ResNet 是一种深度学习架构，主要用于图像分类任务。通过其卓越的图像特征提取能力，能够精准地理解和表征商品图像的关键信息。这一能力被用于构建商品特征库，使得系统能够根据用户点击的商品的图片快速检索出相似商品，还能分析用户偏好，提供个性化和上下文相关的商品推荐，极大地提升了推荐的准确性和用户体验。

ResNet-50 是一个由 49 个卷积层和 1 个全局平均池化层组成的深度神经网络模型，通过跳跃连接将输入直接传递给输出，解决了深度网络中的梯度消失和梯度爆炸问题。ResNet-50 的结构分为多个阶段，每个阶段包含若干个残差块，每个残差块又由多个卷积层组成。在训练过程中，ResNet-50 使用了批量归一化和恒等映射技术来加速收敛和提高模型性能。

1. ResNet 网络结构

ResNet 在 2015 年被提出，在 ImageNet 比赛 classification 任务上获得第一名，因为它“简单与实用”并存，之后很多方法都建立在 ResNet50 或者 ResNet101 的基础上完成的，检测，分割，识别等领域里得到广泛的应用。它使用了一种连接方式叫做“shortcut connection”，顾名思义，shortcut 就是“抄近道”的意思，下面是这个 resnet 的网络结构：

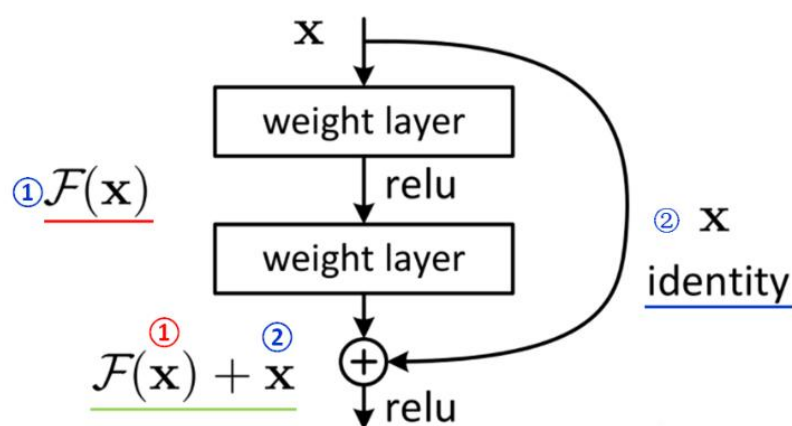


图 5-2 resnet 的网络结构

它对每层的输入做一个 reference (x)，学习形成残差函数，而不是学习一些没有 reference (x) 的函数。这种残差函数更容易优化，能使网络层数大大加深。在上图的残差块中它有二层，如下表达式，其中 σ 代表非线性函数 ReLU。

$$\mathcal{F} = W_2 \sigma(W_1 x)$$

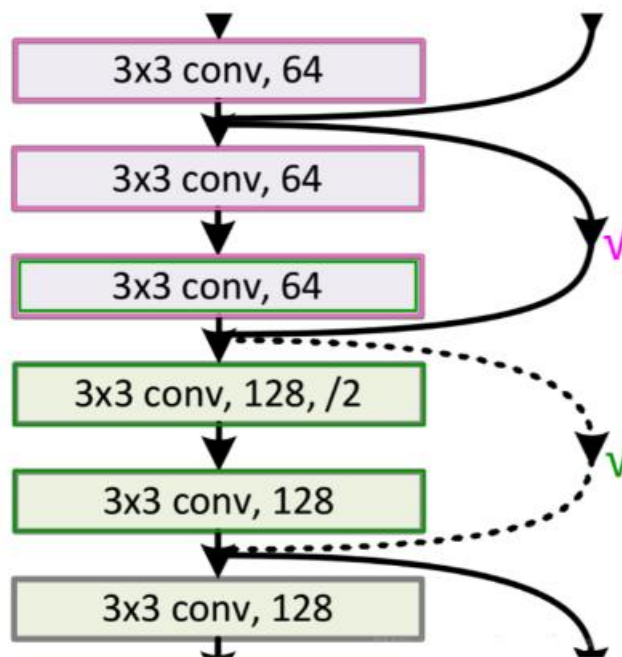
然后通过一个 shortcut，和第 2 个 ReLU，获得输出 y 。

$$y = \mathcal{F}(x, \{W_i\}) + x.$$

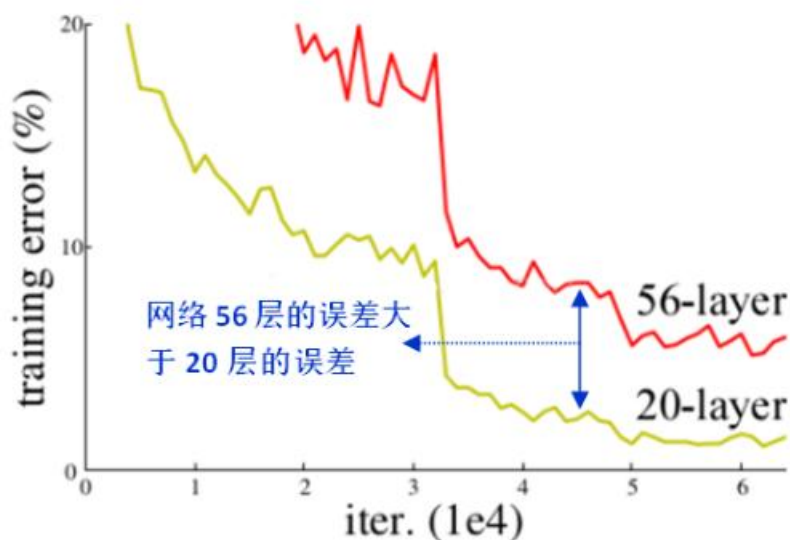
当需要对输入和输出维数进行变化时（如改变通道数目），可以在 shortcut 时对 x 做一个线性变换 W_s ，如下式。

$$y = \mathcal{F}(x, \{W_i\}) + W_s x.$$

然而实验证明 x 已经足够了，不需要再搞个维度变换，除非需求是某个特定维度的输出，如是将通道数翻倍，如下图所示：

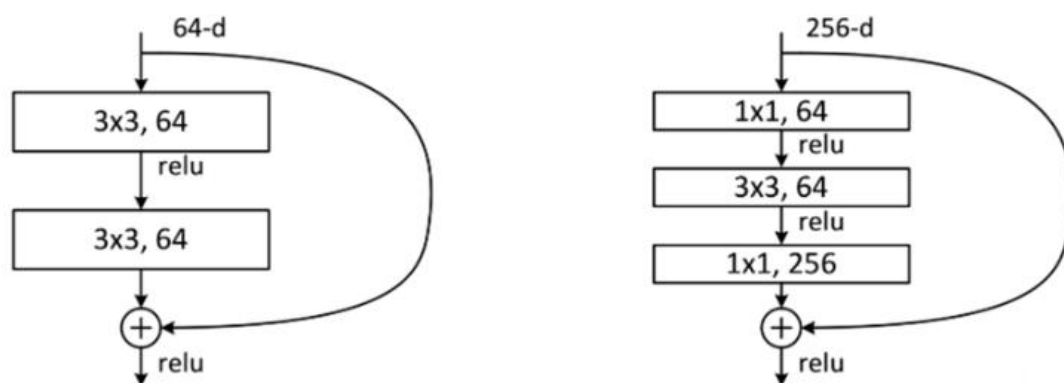


由上图，我们可以清楚的看到“实线”和“虚线”两种连接方式，实线的 Connection 部分（第一个粉色矩形和第三个粉色矩形）都是执行 $3 \times 3 \times 64$ 的卷积，他们的 channel 个数一致，所以采用计算方式： $Y = F(x) + x$ ，虚线的 Connection 部分（第一个绿色矩形和第三个绿色矩形）分别是 $3 \times 3 \times 64$ 和 $3 \times 3 \times 128$ 的卷积操作，他们的 channel 个数不同 (64 和 128)，所以采用计算方式： $y = F(x) + Wx$ 。其中 W 是卷积操作，用来调整 x 的 channel 维度。在计算机视觉里，网络的深度是实现网络好的效果的重要因素，输入特征的“等级”随增网络深度的加深而变高。然而在网络深度不断加深的情况下，梯度弥散/爆炸成为训练深层次的网络的障碍，导致导致网络无法收敛。虽然，归一初始化，各层输入归一化，使得可以收敛的网络的深度提升为原来的十倍。虽然网络收敛了，但网络却开始退化（增加网络层数却导致更大的误差），如下图所示：



由上图可知, 在一个浅层网络的基础上叠加 $y=x$ 的层(称 identity mappings, 恒等映射), 可以让网络随深度增加而不退化。这反映了多层非线性网络无法逼近恒等映射网络。但是, 在深度学习中我们希望有更好性能的网络, 而网络不退化则不是我们的目的。在 ResNet 网络中学习的残差函数是 $F(x) = H(x) - x$, 这里如果 $F(x) = 0$, 那么就是上面提到的恒等映射 ($H(x) = x$)。事实上, ResNet 是 “shortcut connections” 的在 connections 是在恒等映射下的特殊情况, 它没有引入额外的参数和计算的复杂度。假如优化目标函数是逼近一个恒等映射, 而不是 0 映射 ($F(x) = 0$) 或者说恒等映射, 那么学习找到对恒等映射的扰动会比重新学习一个映射函数要容易。

2. 残差块的结构



这两种结构分别针对 ResNet34 (左图) 和 ResNet50/101/152 (右图), 一

般称整个结构为一个“building block”。其中右图又称为“bottleneck design”，目的就是为了降低参数的数目，实际中，考虑计算的成本，对残差块做了计算优化，即将两个 3×3 的卷积层替换为 $1 \times 1 + 3 \times 3 + 1 \times 1$ ，如右图所示。新结构中的中间 3×3 的卷积层首先在一个降维 1×1 卷积层下减少了计算，然后在另一个 1×1 的卷积层下做了还原，既保持了精度又减少了计算量。第一个 1×1 的卷积把 256 维 channel 降到 64 维，然后在最后通过 1×1 卷积恢复，整体上用的参数数目： $1 \times 1 \times 256 \times 64 + 3 \times 3 \times 64 \times 64 + 1 \times 1 \times 64 \times 256 = 69632$ ，而不使用 bottleneck 的话就是两个 $3 \times 3 \times 256$ 的卷积，参数数目： $3 \times 3 \times 256 \times 256 \times 2 = 1179648$ ，差了 16.94 倍。对于常规 ResNet，可以用于 34 层或者更少的网络中，对于 Bottleneck Design 的 ResNet 通常用于更深的如 ResNet50 这样的网络中，可以减少计算和参数量。

3. 和其他模型的对比

依次采用基于形状的 Daisy 模型、基于形状的 HOG 模型、基于颜色的 RGB 直方图模型、以及 ResNet 模型进行实验。可以看出 ResNet 模型表现优异。检索效果如下表所示：

	浏览商品 图片	推荐商品 1	推荐商品 2	推荐商品 3
Daisy				
HOG				
Color				
ResNet				

图 5-3 不同模型对指定图像检索效果对比图

表 5-1 不同模型对 25 类图像检测平均精度对比表

		RGB	HOG	Daisy	ResNet
Mean	MAP	0.614	0.450	0.468	0.944

实现代码:

模型 URL

```
model_urls = {
    'resnet18': 'https://download.pytorch.org/models/resnet18-5c106cde.pth',
    'resnet34': 'https://download.pytorch.org/models/resnet34-333f7ec4.pth',
    'resnet50': 'https://download.pytorch.org/models/resnet50-19c8e357.pth',
    'resnet101': 'https://download.pytorch.org/models/resnet101-5d3b4d8f.pth',
    'resnet152': 'https://download.pytorch.org/models/resnet152-b121ed2d.pth',
}
```

定义 ResidualNet 类

```
class ResidualNet(ResNet):
    def __init__(self, model='resnet152', pretrained=True):
        if model == "resnet18":
            super().__init__(BasicBlock, [2, 2, 2, 2], 1000)
            if pretrained:
                self.load_state_dict(model_zoo.load_url(model_urls['resnet18']))
        elif model == "resnet34":
            super().__init__(BasicBlock, [3, 4, 6, 3], 1000)
            if pretrained:
                self.load_state_dict(model_zoo.load_url(model_urls['resnet34']))
        elif model == "resnet50":
            super().__init__(Bottleneck, [3, 4, 6, 3], 1000)
            if pretrained:
                self.load_state_dict(model_zoo.load_url(model_urls['resnet50']))
        elif model == "resnet101":
            super().__init__(Bottleneck, [3, 4, 23, 3], 1000)
            if pretrained:
                self.load_state_dict(model_zoo.load_url(model_urls['resnet101']))
        elif model == "resnet152":
            super().__init__(Bottleneck, [3, 8, 36, 3], 1000)
            if pretrained:
                self.load_state_dict(model_zoo.load_url(model_urls['resnet152']))

    def forward(self, x):
        x = self.conv1(x)
```

```

x = self.bn1(x)
x = self.relu(x)
x = self.maxpool(x)
x = self.layer1(x)
x = self.layer2(x)
x = self.layer3(x)
x = self.layer4(x) # x after layer4, shape = N * 512 * H/32 * W/32
max_pool = torch.nn.MaxPool2d((x.size(-2), x.size(-1)),
                               stride=(x.size(-2), x.size(-1)), padding=0, ceil_mode=False)
Max = max_pool(x) # max_pool size = N * 512 * 1 * 1
Max = Max.view(Max.size(0), -1) # flatten max_pool to N * 512
avg_pool = torch.nn.AvgPool2d((x.size(-2), x.size(-1)),
                               stride=(x.size(-2), x.size(-1)), padding=0, ceil_mode=False, count_include_pad=True)
avg = avg_pool(x) # avg_pool size = N * 512 * 1 * 1
avg = avg.view(avg.size(0), -1) # flatten avg_pool to N * 512
fc = self.fc(avg) # fc size = N * 1000
output = {
    'max': Max,
    'avg': avg,
    'fc': fc
}
return output

```

计算数据库中所有图像的特征

class ResNetFeat(object):

def make_samples(self, db, verbose=True):

 sample_cache = '{}-{}'.format(RES_model, pick_layer)

try:

 samples = cPickle.load(open(os.path.join(cache_dir, sample_cache), "rb", True))

for sample **in** samples:

 sample['hist'] /= np.sum(sample['hist']) # normalize

if verbose:

 print("Using cache..., config=%s, distance=%s, depth=%s" % (sample_cache, RES_model, pick_layer))

except:

if verbose:

 print("Counting histogram..., config=%s, distance=%s, depth=%s" % (sample_cache, RES_model, pick_layer))

 res_model = ResidualNet(model=RES_model)

 res_model.eval()

if use_gpu:

 res_model = res_model.cuda()

 samples = []

 data = db.get_data()

```

    for d in data.itertuples():
        d_img, d_cls = getattr(d, "img"), getattr(d, "cls")
        img = scipy.misc.imread(d_img, mode="RGB")
        img = img[:, :, ::-1] # switch to BGR
        img = np.transpose(img, (2, 0, 1)) / 255.
        img[0] -= means[0] # reduce B's mean
        img[1] -= means[1] # reduce G's mean
        img[2] -= means[2] # reduce R's mean
        img = np.expand_dims(img, axis=0)
        try:
            if use_gpu:
                inputs = torch.autograd.Variable(torch.from_numpy(img).cuda().float())
            else:
                inputs = torch.autograd.Variable(torch.from_numpy(img).float())
            d_hist = res_model(inputs)[pick_layer]
            d_hist = d_hist.data.cpu().numpy().flatten()
            d_hist /= np.sum(d_hist) # normalize
            samples.append({
                'img': d_img,
                'cls': d_cls,
                'hist': d_hist
            })
        except:
            pass
    cPickle.dump(samples, open(os.path.join(cache_dir, sample_cache), "wb"), True)

    return samples

```

```

from flask import Flask, request, jsonify
import os
import pickle
import pymysql
import torch
from torchvision import models, transforms
from PIL import Image
from sklearn.metrics.pairwise import cosine_similarity
import requests

```

```
app = Flask(__name__)
```

```
# 数据库连接配置
```

```
db_config = {
    'host': 'localhost',
    'user': 'root',

```

```

        'password': 'Ly030704',
        'database': 'mall',
        'charset': 'utf8mb4',
        'cursorclass': pymysql.cursors.DictCursor
    }

# ResNet50 模型加载和特征提取
resnet_model = models.resnet50(pretrained=True)
resnet_model.eval()
transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.2
25])),
])

# 提取图片特征向量
def extract_resnet_feature(img_url):
    try:
        response = requests.get(img_url, stream=True, verify=False)
        response.raise_for_status()
        img = Image.open(response.raw).convert('RGB')
        img = transform(img).unsqueeze(0)
        with torch.no_grad():
            features = resnet_model(img)
        return features.squeeze().numpy()
    except Exception as e:
        print(f"Error opening image from URL {img_url}: {e}")
        return None

# 保存特征向量到 pickle 文件
def save_feature_to_pickle(good_id, feature):
    filename = f"features/{good_id}.pkl"
    with open(filename, 'wb') as f:
        pickle.dump(feature, f)

# 加载特征向量从 pickle 文件
def load_feature_from_pickle(good_id):
    filename = f"features/{good_id}.pkl"
    try:
        with open(filename, 'rb') as f:
            feature = pickle.load(f)
        return feature

```

```

except FileNotFoundError:
    return None

# 查询用户浏览过的商品 ID
def get_viewed_goods(user_id):
    connection = pymysql.connect(**db_config)
    try:
        with connection.cursor() as cursor:
            sql = "SELECT goodid FROM guess WHERE userid = %s"
            cursor.execute(sql, (user_id,))
            result = cursor.fetchall()
            viewed_goods = [row['goodid'] for row in result]
    finally:
        connection.close()
    return viewed_goods

# 计算相似度
def compute_similarity(features1, features2):
    similarity = cosine_similarity([features1], [features2])
    return similarity[0][0]

# 从数据库中获取商品图片链接
def fetch_img_url_from_db(good_id):
    connection = pymysql.connect(**db_config)
    try:
        with connection.cursor() as cursor:
            sql = "SELECT imgs FROM good WHERE id = %s"
            cursor.execute(sql, (good_id,))
            result = cursor.fetchone()
            if result:
                img_url = result['imgs']
                return img_url # 直接返回从数据库中获取的图片链接
            else:
                return None
    finally:
        connection.close()

# 推荐相似商品给用户
def recommend_similar_goods(user_id, top_n_per_image=10):
    viewed_goods = get_viewed_goods(user_id)
    all_recommendations = set()

    for good_id in viewed_goods:
        # 先尝试从本地加载特征向量

```

```

features1 = load_feature_from_pickle(good_id)

if features1 is None:
    # 如果本地没有特征向量，则重新提取并保存
    img_url = fetch_img_url_from_db(good_id) # 从数据库获取图片链接
    features1 = extract_resnet_feature(img_url)
    if features1 is not None:
        save_feature_to_pickle(good_id, features1)

if features1 is not None:
    connection = pymysql.connect(**db_config)
    try:
        with connection.cursor() as cursor:
            sql_all = "SELECT id, imgs FROM good WHERE id != %s"
            cursor.execute(sql_all, (good_id,))
            all_goods = cursor.fetchall()

            for row in all_goods:
                features2 = load_feature_from_pickle(row['id'])
                if features2 is None:
                    img_url = row['imgs']
                    features2 = extract_resnet_feature(img_url)
                    if features2 is not None:
                        save_feature_to_pickle(row['id'], features2)
            )

            if features2 is not None:
                similarity = compute_similarity(features1, features2)

            all_recommendations.add((row['id'], similarity,
            row['imgs']))

    finally:
        connection.close()

# 过滤掉用户已购买过的商品
connection = pymysql.connect(**db_config)
try:
    with connection.cursor() as cursor:
        sql_purchased = "SELECT goodid FROM guess WHERE userid = %s"
        cursor.execute(sql_purchased, (user_id,))
        purchased_goods = set(row['goodid'] for row in cursor.fetchall())
    )

```

```

    finally:
        connection.close()

    # 去重和排序
    unique_recommendations = []
    seen_ids = set()
    for rec in sorted(all_recommendations, key=lambda x: x[1], reverse=True)
:
        if rec[0] not in purchased_goods and rec[0] not in seen_ids:
            unique_recommendations.append({'id': rec[0], 'similarity': rec[
1], 'img_url': rec[2]})
            seen_ids.add(rec[0])

        if len(unique_recommendations) >= top_n_per_image * len(viewed_good
s):
            break

    return unique_recommendations[:top_n_per_image * len(viewed_goods)]

# Flask 路由
@app.route('/recommend_goods', methods=['GET'])
def recommend_goods():
    user_id = request.args.get('user_id')
    if not user_id:
        return jsonify({'error': 'Missing user_id parameter'}), 400

    try:
        user_id = int(user_id)
    except ValueError:
        return jsonify({'error': 'Invalid user_id parameter'}), 400

    try:
        recommendations = recommend_similar_goods(user_id)
        recommended_ids = [rec['id'] for rec in recommendations]
        return jsonify({'recommended_ids': recommended_ids})
    except Exception as e:
        return jsonify({'error': str(e)}), 500

if __name__ == '__main__':
    app.run(debug=True, port=8081)

```

其中 ResidualNet 继承自 ResNet 并根据传入的模型类型 (resnet18,

resnet34, resnet50, resnet101, resnet152) 初始化相应的层数。

forward 方法定义了模型的前向传播，包括卷积层、池化层以及全连接层的操作。

特征提取：

ResNetFeat 类中的 make_samples 方法用来计算数据库中所有图像的特征。读取图像并预处理，然后通过 ResidualNet 模型提取特征。

提取的特征包括 max_pool 和 avg_pool 后的特征，以及全连接层的输出。

第 5 章 项目实施与使用

5.1 项目实施

开发工具：

IDE: IntelliJ IDEA 用于后端开发，微信开发者工具用于前端开发

版本控制: Git 和 GitHub 用于代码管理和协作

包管理工具: npm 和 Maven 分别用于管理前端和后端依赖

开发环境：

操作系统: Ubuntu 20.04 LTS 用于服务器部署，Windows 11 用于本地开发

前端技术: 微信小程序框架、HTML5、CSS3、JavaScript

后端技术: Spring Boot 用于业务逻辑处理，Flask 用于虚拟试穿功能

数据库: MySQL 用于存储核心数据，Redis 用于缓存

中间件: RabbitMQ 用于订单异步处理

项目部署与配置

1. 准备服务器：

确保服务器上安装了 Java、Python、MySQL 和 Redis。

配置防火墙，开放必要的端口，如 HTTP(S)、MySQL 和 Redis 端口。

2. 部署后端服务：

将 Spring Boot 项目打包为 JAR 文件，并上传到服务器。

在服务器上运行 `java -jar` 命令启动 Spring Boot 服务。

将 Flask 项目打包，并使用 Gunicorn 等 WSGI 服务器进行部署。

3. 配置数据库：

在 MySQL 中创建所需数据库和表结构。

导入初始数据，包括用户、商品和订单等基础数据。

4. 部署前端小程序：

使用微信开发者工具，将小程序代码上传并发布。

确保小程序的 API 地址正确指向已部署的后端服务。

5. 配置中间件：

安装并配置 RabbitMQ，用于处理订单异步消息队列。

5.2 系统使用手册

1. 用户操作说明：

(1) 注册与登录：

用户可以通过微信授权登录进入系统。

完成注册后，可以进行个人信息的修改和管理。

(2) 商品浏览与购买：

用户可以浏览首页推荐商品，进行商品筛选和排序。

点击商品进入详情页，选择规格和数量，加入购物车或直接购买。

选择收货地址并进行虚拟支付，完成订单。

(3) 同城购物：

浏览同城商品，选择快递配送或到店自取。

确认订单并选择配送方式，完成支付。

(4) 虚拟试鞋与试衣：

使用虚拟试鞋功能，通过 AR 技术查看试穿效果。

使用虚拟试衣功能，通过上传照片试穿不同衣物。

(5) 客服聊天：

通过实时客服聊天功能，与客服进行在线沟通。

解决购物过程中遇到的问题。

(6) 直播功能：

观看商家直播，了解商品详情和优惠信息。

参与直播互动，享受购物乐趣。

2. 商家操作说明：

(1) 商品管理：

上传商品图片和信息，进行商品的上架和下架操作。

管理商品分类和库存，查看商品评价。

(2) 订单管理：

查看并处理用户订单，管理订单的发货和退货。

处理订单的支付和结算，确保订单正常完成。

(3) 发起直播：

通过腾讯云服务发起直播，展示商品并与用户互动。

管理直播间信息，确保直播顺利进行。

3. 管理员操作说明：

(1) 用户管理：

查看和管理用户信息，处理用户反馈和投诉。

管理用户权限，确保系统安全。

(2) 商家管理：

审核和管理商家信息，处理商家申请和反馈。

管理商家权限，确保商家行为合规。

5.3 项目效果



图 6-1 商城首页



图 6-2 商品列表

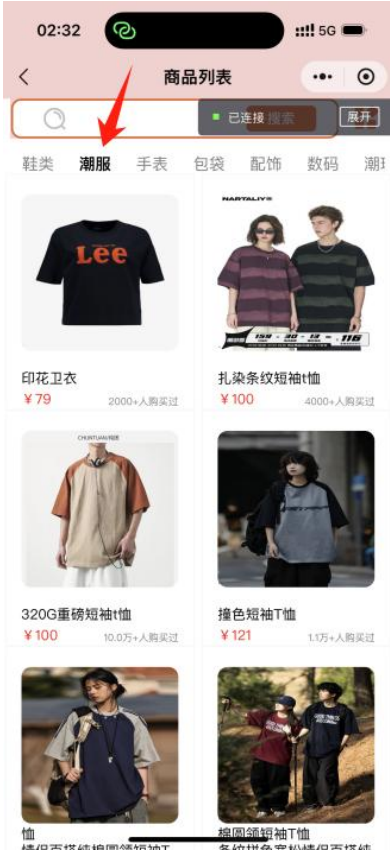


图 6-3 商品列表中的分类



图 6-4 商品详情页



图 6-5 商品细节页



图 6-6 展开商品的所有评论



图 6-7 选择规格

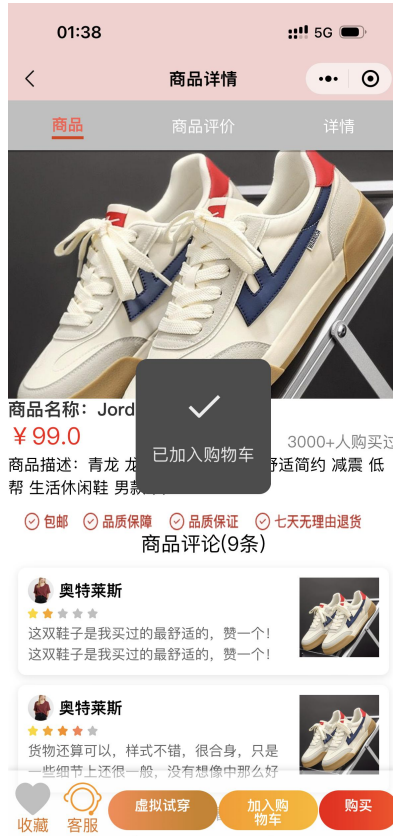


图 6-8 加入购物车成功



图 6-9 地址管理页

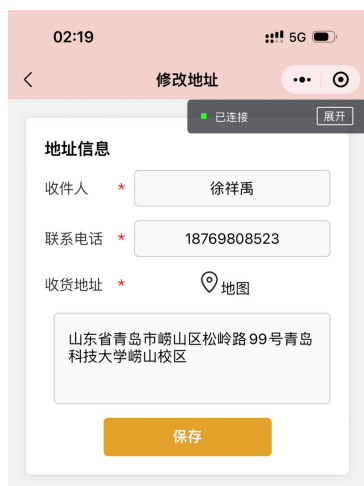


图 6-10 新增地址

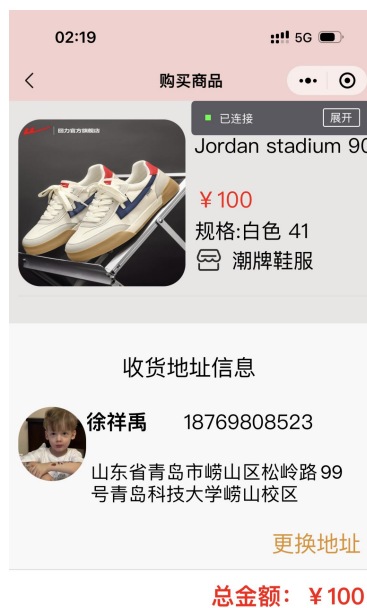


图 6-11 提交订单



图 6-12 商品分类



图 6-13 虚拟试穿鞋子



图 6-14 原图



图 6-15 虚拟试穿上的鞋子



图 6-16 展示鞋的提示点



图 6-17 客服聊天



图 6-18 聊天页面



图 6-19 查看新品



图 6-20 筛选条件

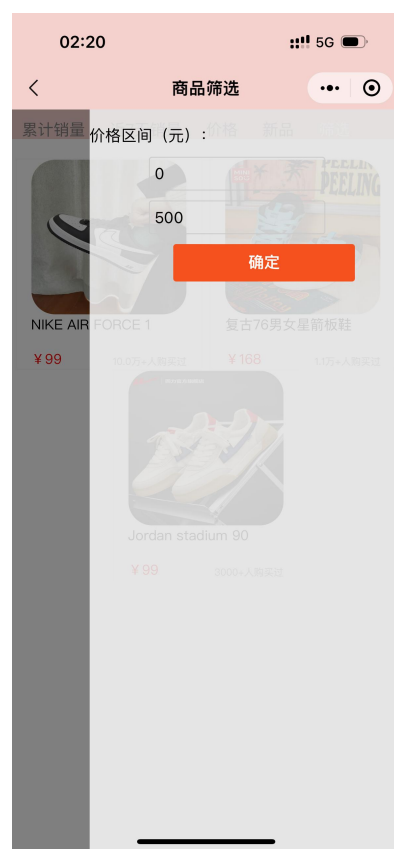


图 6-21 设置区间

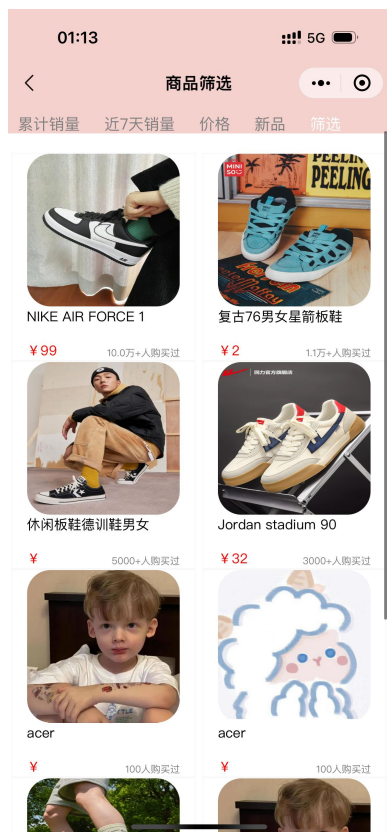


图 6-22 筛选后的商品



图 6-23 进入直播



图 6-24 观看直播

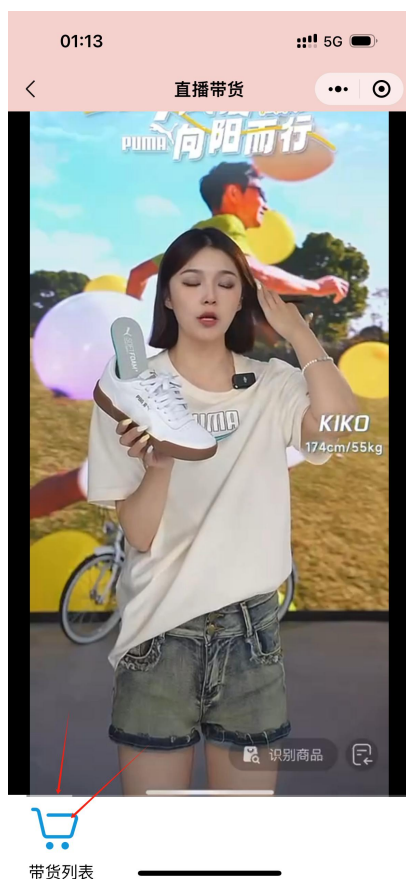


图 6-25 点击带货列表

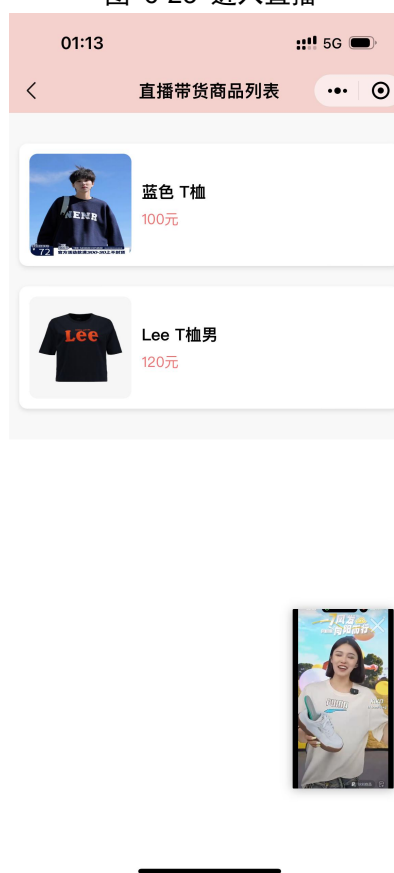


图 6-26 列表详情



图 6-27 购买带货商品



图 6-28 同城购物页面



图 6-29 显示商家距离

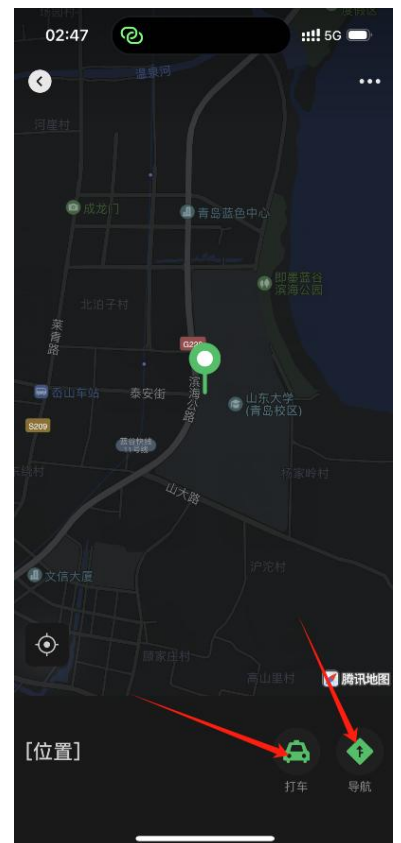


图 6-30 提供到店方式

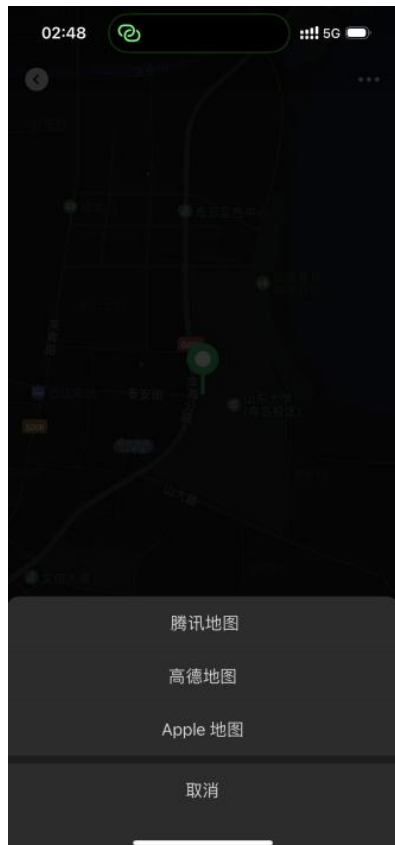


图 6-31 选项



图 6-32 到店方式

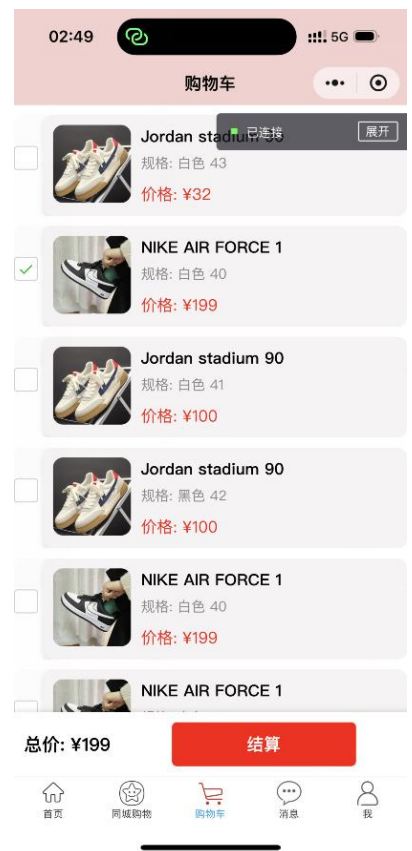


图 6-33 购物车界面



图 6-34 删除购物车中的商品



图 6-35 消息列表界面



图 6-36 个人中心界面

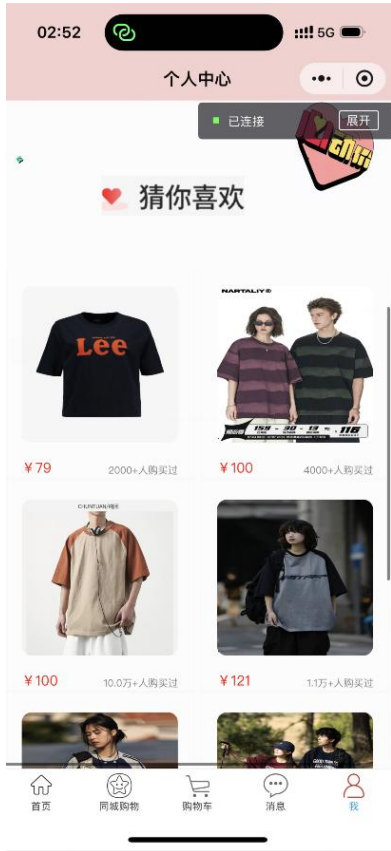


图 6-37 推荐界面



图 6-38 修改个人信息



图 6-39 全部订单



图 6-40 待收货的订单



图 6-41 待评价的订单



图 6-42 已完成的订单

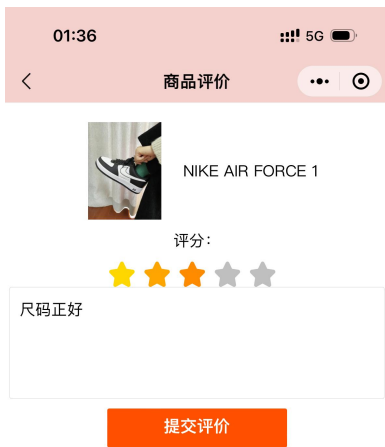


图 6-43 评价界面



图 6-44 虚拟试衣界面



图 6-45 虚拟试衣效果

第 6 章 系统测试

6.1 测试目的

本测试旨在确保小程序的质量和稳定性，测试小程序的功能、性能和用户体验，验证其是否满足开发需求，并发现并修复潜在问题。

6.2 测试环境

测试智慧商城小程序时，使用不同配置的移动设备打开微购小程序测试各项功能是否正常。测试手机的配置信息如表所示。

手机型号	操作系统	处理器	运行内存	微信版本
iPhone 15	iOS17	A16	6GB	8.0.48
iPhone 15 pro	iOS17	A17 Pro	8GB	8.0.45
小米 13	Android 13	高通骁龙 8 Gen 2	8GB	8.0.21
华为 P60	鸿蒙 3.1	高通骁龙 8+4G	8GB	8.0.48

6.3 功能测试

1. 商品分类查询与搜索功能测试

用户在智慧商城中使用分类查询或者搜索商品功能的测试，包括商品的分类检索以及商品的查询功能测试。描述见下表所示。

用例编号	预置条件	步骤	预期结果	测试结果
w-user-01001	无	1. 点击底部“分类”按钮； 2. 测试一级分类是否正常； 3. 点击一级分类名称，测试是否加载出	1. 进入分类展示界面； 2. 左侧边栏展示完整的一级分类列表； 3. 二级分类正确	通过

		二级分类; 4. 点击二级分类名称, 测试是否正确加载该分类商品;	加载; 4. 分类商品正确显示;	
w-user-01002	用户已进入分类界面	1. 点击顶部搜索框, 测试是否能正常输入搜索关键词; 2. 点击“搜索”按钮后, 测试是否显示搜索结果	1. 搜索框输入正常; 2. 正确显示关键词的输入结果;	通过

2. 购物车管理功能测试

测试用户在智慧商城小程序中使用购物车的功能是否正常, 具体为添加商品到购物车、从购物车结算及编辑购物车商品等功能, 详细描述见下表。

用例编号	预置条件	步骤	预期结果	测试结果
w-user-02001	用户静默登录成功	1. 随意添加若干件商品到购物车, 测试是否能正常加入; 2. 点击底部“购物车”按钮进入购物车管理界面; 3. 滑动某商品, 选择删除, 测试是否能删除商品; 4. 修改某个商品的数量, 测试数量修改是否成功; 5. 选中购物车中的部分商品结算, 测试订单中商品是否正确;	1. 商品能正确加入到购物车; 2. 能够打开小程序的购物车管理界面; 3. 能够正常移除商品; 4. 能够修改某商品的数量; 5. 选中的部分商品与订单中的商品信息一致;	通过

3. 订单查询功能测试

测试智慧商城小程序中订单查询功能，具体为订单的分类查询及订单的详细信息查询，详细描述见下表。

用例编号	预置条件	步骤	预期结果	测试结果
w-user-03 001	用户静默登录成功	1. 点击底部“我的”，进入个人中心； 2. 点击“全部订单”按钮； 3. 分别点击“待付款”、“待发货”“待收货”、“已完成”按钮，测试是否能正确显示订单信息； 4. 点击某一订单，测试是否显示订单详情；	1. 正确显示个人中心界面； 2. 进入订单列表显示界面； 3. 能够正确显示当前条件下的订单； 4. 能够正确显示该订单详情；	通过

4. 下单与结算功能测试

测试智慧商城小程序创建订单与结算订单功能，具体为从购物车创建订单、从商品购买页面创建订单、订单的支付结算、订单支付后状态改变和订单支付金额核算等功能。详细描述见表。

用例编号	预置条件	步骤	预期结果	测试结果
w-user-04 001	用户静默登录成功	1. 随意挑选一款商品，点击“立即购买”进入订单创建界面； 2. 从购物车选择若干商品，点击“结算”按钮； 3. 点击创建订单界面顶部的地址栏，测试新增收货地址或者选择已有收货地址功能是否正常；	1. 能够正确显示订单创建界面； 2. 能够正确显示订单创建界面； 3. 选择已有收货地址功能正常，能够新增收货地址； 4. 创建订单界面	通过

		4. 创建订单界面显示当前商品可用优惠券, 测试优惠券使用是否合法 5. 点击底部“提交订单”按钮, 测试是否创建成功并进入到待付款界面	显示的可用优惠券合法; 5. 获取到订单号并在待付款中显示订单;	
w-user-04002	已创建订单但未支付	1. 点击“我的”按钮, 点击“待付款”按钮, 进入到待付款订单界面; 2. 点击某个订单下方的“支付”按钮, 测试是否支付成功; 3. 完成支付后, 进入“待发货”订单界面, 测试订单是否支付成功:	1. 成功进入待付款订单界面 2. 显示支付成功 3. 支付后订单状态变更成功	通过

5. AR 试鞋功能测试

测试智慧商城小程序 AR 试鞋功能, 用户点击鞋子商品详情页面的“虚拟试穿”按钮, 上传或拍摄脚部照片, 系统展示试鞋效果。

用例编号	预置条件	步骤	预期结果	测试结果
w-user-05001	用户授权访问相机和存储。	1. 打开小程序, 进入支持虚拟试鞋的商品详情页。 2. 点击“虚拟试鞋”按钮。 3. 允许小程序访问相机和存储权限 (如果未授权)。 4. 将手机对准脚部拍摄或上传已有脚部照片。	1. 用户能顺利进入虚拟试鞋功能页面。 2. 相机和存储权限申请正常, 用户能授权访问。 3. 系统能准确识别脚部, 并展示虚拟试穿效果	通过

		5. 系统加载虚拟试鞋功能，展示虚拟试鞋效果。 6. 更换不同的鞋子款式，观察虚拟试穿效果。 7. 选择一款满意的鞋子，添加到购物车。	果。 4. 不同鞋子款式的虚拟试穿效果展示正确。 5. 用户能顺利将选择的鞋子添加到购物车。	
--	--	---	--	--

6. AI 换衣功能测试

测试智慧商城小程序 AI 换衣功能，用户点击衣服商品详情页面的“虚拟试穿”按钮，上传或拍摄全身照片，系统展示试衣效果。

用例编号	预置条件	步骤	预期结果	测试结果
w-user-06001	用户授权访问相机和存储。	1. 打开小程序，进入支持 AI 换衣的商品详情页。 2. 点击“AI 换衣”按钮。 3. 允许小程序访问相机和存储权限（如果未授权）。 4. 拍摄全身照片或上传已有全身照片。 5. 系统加载 AI 换衣功能，展示虚拟试穿效果。 6. 更换不同的衣服款式，观察虚拟试穿效果。 7. 选择一款满意的衣服，添加到购物车。	1. 用户能顺利进入 AI 换衣功能页面。 2. 相机和存储权限申请正常，用户能授权访问。 3. 系统能准确识别用户全身，并展示虚拟试穿效果。 4. 不同衣服款式的虚拟试穿效果展示正确。 5. 用户能顺利将选择的衣服添加到购物车。	通过

7. 客服聊天功能测试

测试智慧商城小程序客服聊天功能，用户打开客服聊天界面，发送消息，客服接收并回复。

用例编号	预置条件	步骤	预期结果	测试结果
w-user-07 001	无	1. 用户选择商品并选择“联系客服”按钮。 2. 在聊天窗口输入消息并发送。 3. 客服接收消息并进行回复。 4. 用户查看回复，并继续与客服进行对话。 5. 结束对话，关闭聊天窗口。	1. 用户能顺利进入客服聊天窗口。 2. 消息发送和接收正常，无延迟或丢失。 3. 客服能及时回复用户消息，提供帮助。 4. 聊天结束后窗口正常关闭，消息记录保存。	通过

8. 直播模块功能测试

测试智慧商城小程序直播模块功能，商家发起直播，用户进入直播页面观看直播视频并与商家互动。

用例编号	预置条件	步骤	预期结果	测试结果
w-user-08 001	家已开启直播，用户网络连接正常，带宽满足视频直播要求。	1. 用户进入小程序并选择“观看直播”按钮。 2. 选择正在进行的直播，点击进入。 3. 观看直播内容。 4. 在直播互动区输入评论或提问。	1. 用户能顺利进入直播观看页面。 2. 直播视频流畅播放，清晰度良好，无卡顿。 3. 评论和提问	通过

		5. 商家在直播中回复用户评论或提问。 6. 购买直播中推荐的商品，添加至购物车。 7. 结束直播，退出观看页面。	互动正常，商家及时回复。 4. 直播推荐商品能正常添加至购物车。 5. 直播结束后，用户能正常退出观看页面。	
--	--	---	--	--

9. 同城购物功能测试

测试智慧商城小程序同城购物功能，用户浏览同城购物页面商品，选择快递配送或到店自取方式，下单并支付。

用例编号	预置条件	步骤	预期结果	测试结果
w-user-09001	系统已获取用户位置，并根据位置推荐同城商品。 同城商家已开设店铺，商品信息已上传。	1. 用户进入小程序并选择“同城购物”按钮。 2. 系统根据用户位置推荐同城商品列表。 3. 用户选择商品并查看详情。 4. 选择配送方式（快递配送或到店自取）。 5. 若选择到店自取，系统推荐合适的网约车或外卖服务。 6. 用户确认购买，选择支付方式并完成支付。 7. 查看订单详情，跟踪配送状态或到店自取信息。	1. 用户能顺利进入同城购物页面。 2. 系统推荐的同城商品准确，无误。 3. 商品详情页面显示正常，信息完整。 4. 用户能顺利选择配送方式并完成购买。 5. 到店自取功能正常，网约车和外卖服务推荐准确。	通过

			6. 订单详情和配送状态显示正确。	
--	--	--	-------------------	--

6.4 性能测试

在智慧商城小程序中，后台接口可分为公开接口和非公开接口。首页中展示的 banner、主题、商品和分类等都从公开 API 请求数据，为了提高响应速度，公开接口不需要签名。表 6-1 为利用 ApacheJMeter 模拟用户并发访问部分公开接口的测试结果。

表 6-1 模拟用户并发访问部分接口的测试结果

并发数	总请求数	平均值	最小值	最大值	99%响应时间	吞吐量	异常占比
500	62997	436	4	1349	815	858/s	0%
1000	390093	773	3	2135	1424	897/s	0%
1500	220123	1095	5	3317	2258	860/s	0%
2000	56835	1414	3	3161	2517	850/s	0%
2200	70315	1530	4	3205	2734	855/s	0%
2500	61075	2187	11	4331	3403	824/s	0%
3000	149969	2472	3	5248	4528	838/s	0%

从表 8-2 所示测试结果得出结论，接口平均响应时间与并发数表现为正相关关系。并发数为 1000 时，系统吞吐量为 897，99%用户响应时间小于 2 秒。当并发数增长到 1500 时，接口平均响应时间小于 2 秒，然而 99%用户响应时间大于 2 秒。测试结果表明，1000 位用户并发访问时，后台服务器能在 2 秒内正确响应，符合需求分析的执行效率指标。

6.5 总结

通过以上性能测试和功能测试，我们确认智慧商城小程序具备良好的性能表现和功能完整性，能够为用户提供稳定流畅的购物体验。系统在高并发情况下运行稳定，响应迅速，所有功能模块均能正常使用，满足用户需求。

第 7 章 总结

7.1 克服的困难

1. **微信小程序对非商户号的限制：**由于我们没有商户号，无法使用支付接口和直播等功能。

问题：无法通过微信支付接口进行支付和直播功能的实现。

解决方案：对于直播功能，我们使用了腾讯云服务，利用其推流地址和视频接收地址来实现直播功能。

2. **用户体验：**为了确保用户有良好的体验，我们反复优化前端页面和交互设计，进行了多轮测试和反馈修改，最终达到了理想的用户体验效果。

问题：初期用户界面设计缺乏用户友好性，交互体验不佳。

解决方案：我们进行了多轮用户测试，收集了大量用户反馈，并根据反馈不断改进界面设计和交互逻辑。同时，采用了前端性能优化技术，如图片懒加载、代码分片等，提升了页面加载速度。

3. **复杂功能实现：**实现 AR 试鞋和 AI 换衣功能、实时客服聊天、同城购物等复杂功能，需要掌握先进的技术和算法。通过查阅大量文献资料 and 不断实验，我们最终成功实现了这些功能。

问题：AR 和 AI 技术的实现和集成。

解决方案：我们选用了改进的第三方技术，如 VisionKit 和 VOfitFusion 模型，来实现 AR 试鞋和 AI 换衣功能。深入学习了这些技术的原理和应用方法，通过多次实验和调试，最终实现了这些功能的稳定运行。

4. **性能优化：**面对大量用户访问和高并发请求，系统性能优化成为一个关键问题。我们通过优化数据库查询、使用 Redis 缓存、进行代码优化等手段，提高了系统的响应速度和稳定性。

问题：高并发情况下，数据库查询和写入操作的性能瓶颈。

解决方案：我们采用了数据库读写分离策略，将读操作分散到多个从库，减轻主库压力。此外，使用 Redis 缓存热点数据，减少数据库访问频率。对于复杂查询，我们优化了 SQL 语句，并创建了必要的索引。

5. **技术整合**: 由于项目涉及多个技术栈, 包括微信小程序框架、Spring Boot、Flask、MyBatis、Redis 等, 我们通过详细的技术方案设计和不断的调试, 最终实现了技术的无缝整合。

问题: 不同技术栈之间的接口设计和数据传递。

解决方案: 我们设计了统一的 API 接口标准, 以确保各模块之间的数据传输顺畅。

7.2 升级演进

为了保持智慧商城小程序的竞争力和用户满意度, 我们规划了以下升级演进方向:

功能扩展: 增加更多实用功能, 如积分系统、社交分享等, 提升用户的互动性和粘性。

智能推荐: 优化推荐算法, 结合用户的浏览历史、购买行为和社交关系, 提供更加精准的商品推荐。

多平台支持: 扩展支持其他小程序平台和移动应用, 覆盖更多用户群体。

数据分析: 引入大数据分析技术, 深入分析用户行为和市场趋势, 提供数据驱动的运营决策支持。

安全升级: 持续加强系统的安全防护措施, 保障用户数据和交易安全。

7.3 商业推广

在商业推广方面, 我们将采取以下策略:

线上推广: 通过社交媒体、搜索引擎优化 (SEO)、内容营销等手段, 提升智慧商城小程序的品牌知名度和用户流量。

线下推广: 与实体店合作, 通过联合促销活动和广告宣传, 吸引更多线下用户。

用户激励: 实施用户激励计划, 如邀请好友奖励、消费积分兑换、优惠券发放等, 激发用户的使用和分享热情。

合作伙伴: 与物流公司、支付平台、知名品牌等建立战略合作, 提供更优质的服务和资源。

持续改进：根据用户反馈和市场变化，持续改进和优化产品，保持竞争优势。

7.4 感悟

智慧商城项目中，我主要负责系统架构设计、数据库设计与实现以及后端开发等关键任务。通过系统架构设计，设计了适合项目需求的总体架构，并通过绘制系统架构图详细描述了各层次的关系和功能模块。在数据库设计方面，使用了 MyBatis 进行数据操作，并结合 MySQL 和 Redis 优化了数据存储和访问性能。在后端开发中，使用 Spring Boot 框架实现了用户管理、商品管理和订单管理等核心功能，并与前端的 API 接口进行了有效整合。这些经历不仅加强了 my 的技术能力，还培养了我 在系统设计和项目管理方面的能力，为今后在软件开发领域的进一步探索奠定了坚实基础。

参考文献

[1] 谭台哲,陈宏才,杨卓.VTON-FG:通过图像边缘轮廓特征引导的虚拟试衣网络[J/OL].计算机工程与应用,1-11[2024-07-07].<http://kns.cnki.net/kcms/detail/11.2127.TP.20240624.1748.013.html>.

[2] 张文悦,贾子彦,李青,等.基于 ResNet 和 UNet 的胃癌病理图像诊断系统[J/OL].河北大学学报(自然科学版):1-8[2024-07-07].<http://kns.cnki.net/kcms/detail/13.1077.n.20240628.1604.026.html>.

[3] <https://blog.csdn.net/universsky2015/article/details/136835215>